



Travaux d'Etudes et de Recherches

Teacher: Emmanuel Fleury

White-box AES : implementation and attacks

Authors:

Aymen ALI YAHIA, Nathan DOÏMO

October 5, 2026

Contents

1	Introduction and goals	2
2	Creation of the AES white-box	3
2.1	Implementation overview	3
2.2	A quick reminder about AES	5
2.3	Table-based implementation	6
2.3.1	Changing order of operations	6
2.3.2	Introducing T-boxes	7
2.3.3	T_{y_i} tables	8
2.3.4	XOR Tables	8
2.3.5	Combining tables	9
2.4	White-box protection	10
2.4.1	Internal encoding: masking	10
2.4.2	Mixing bijections	13
3	Hacking the box	14
3.1	Differential Fault Analysis (DFA)	14
3.2	Differential Computation Analysis (DCA)	16
4	Conclusion	19
A	AES encryption using white-box lookup tables	22
B	Test script to validate white-box implementation	23
C	Extract of the proof-of-concept	24

1 Introduction and goals

The security of a cryptosystem holds entirely within the secret key it uses.

Kerckhoffs's principle

Kerckhoffs's principle, created in the 19th century, is a foundational concept of modern cryptography. It states that the reliability of a cryptosystem doesn't rely on the secrecy of its implementation or design but only on the secret key used to encrypt [9]. Otherwise speaking, it implies that any adversary will eventually get complete knowledge of the cryptosystem, from the algorithm used to the most minute details of its implementation. Following this principle, security through obscurity methods like steganography only provide temporary protection.

Nowadays, contemporary computer systems are highly complex and protean, increasing significantly the attack surface available for software vulnerabilities and malicious software. It is then pretty common for a company to assume their software will be run in an environment that cannot be trusted, like user devices.

This leads to a problem: we wish to allow users to decrypt protected data without giving them explicit access to the secret key. Moreover, we would like the users to not be able to recover it in the decryption implementation itself.

This setting is known as the white-box model: a cryptographic model in which the adversary has full control over the environment where our software is executed [10]. This includes entire access to memory, registers, the ability to observe and modify internal computations, etc. It is one of the strongest modern adversarial models considered in cryptography.

White-box cryptography is generally used in DRM (Digital Rights Management) software to protect access to content such as video-streaming or video games. Its main purpose is for a user to access the content they paid for, but giving them the key would enable them to distribute it, granting unrestricted access to that content without requiring payment [6]. Denuvo and Widevine are contemporary examples of such DRM software.

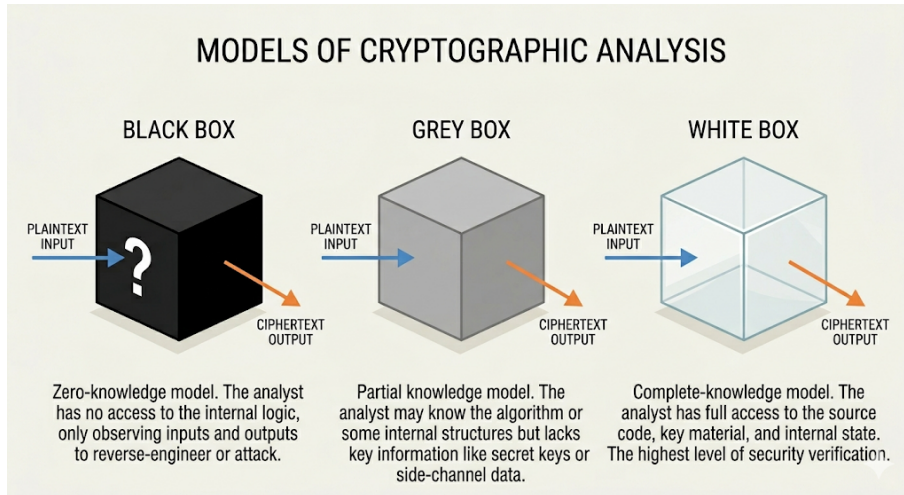


Figure 1: Different cryptographic analysis models (Made with Gemini Nano Banana 2)

Theoretically, creating a white-box for any block cipher is easy: the only thing we need is a look-up table mapping every possible plaintext to the corresponding ciphertext encrypted with a specific key. Such a trivial construction would allow any user to encrypt or decrypt any plaintext without any knowledge of the key used. Furthermore, recovering the key from the table would be the same as solving the key-recovery problem for the cipher, which is computationally hard for *actual* ciphers.

However, this approach is entirely impossible to do due to the memory requirements. Realistically, a block-based cipher using blocks of length l would create a look-up table of 2^l possible entries, each one being mapped to a l -bit ciphertext. In our case, for AES-128, the blocks are

128-bits long meaning 2^{128} entries of 128 bits. The table would hence be made of :

$$2^{128} \times 128 = 2^{135} \text{bits} \approx 5 \times 10^{39} \text{bytes}$$

In real life units, the total amount of data stored worldwide is estimated to be around 10^{24} bytes. The trivial lookup table would then be 10^{15} times larger than all data ever stored by humanity [12], rendering this construction simply impossible.

Despite the memory limitations, representing cryptographic computations using lookup tables is still a central idea in white-box cryptography. The real challenge is more about making an implementation that's compact while preserving the security of the key within the cryptographic system.

The first section will focus on the construction of a white-box implementation of AES-128. We'll first briefly review the basics of the AES-128 algorithm. Then, we'll implement a simple table-based version that won't withstand any attack yet. Finally, we will introduce multiple transformations and encodings aiming at protecting the embedded key from very basic attacks.

In the second section, we'll switch roles and attack our white-box AES. We'll present some classical and modern attacks against white-box protections and attempt to recover the key from different versions of our implementation. This analysis will highlight the limitations of the white-box model and will illustrate how the latter is quite difficult to achieve.

2 Creation of the AES white-box

2.1 Implementation overview

This project aims to design and implement a simple Proof Of Concept (PoC) of a white-box AES-128 encryption system. The objective is to simulate a realistic DRM-like scenario where a manufacturer embeds critical information that must be kept secret while used by its software. Figure 2 below illustrates the relation and interactions between the files of the PoC. It also highlights the two distinct parts of the system: the manufacturer's section (blue) and the user's section (green).

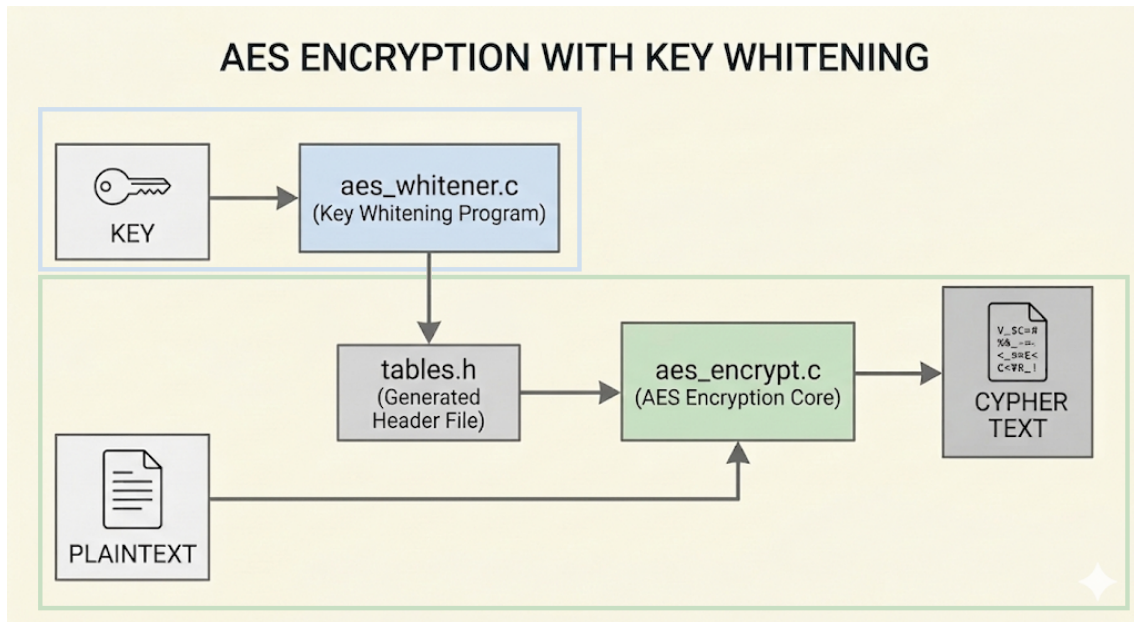


Figure 2: Diagram of the white-box POC implementation (Made with Gemini Nano Banana 2)

Trusted phase (manufacturer)

The first phase of the simulated DRM software corresponds to the "trusted" or "manufacturer" environment. Represented by the blue section in Figure 2, it is a setting in which the secret key is known and can be safely manipulated. The only binary involved in this part is `aes_whitener`.

It is by far the most important and most complex piece of the framework. It is responsible for generating a C header file containing all lookup tables used for AES-128 encryption. The structure and properties of which will be discussed in the later section.

This phase does not fall within the white-box threat model per se: the secret key is explicitly used in the computations and no protection is expected at this stage, as it must be provided as input to the tables generation process. Moreover, an attacker is not assumed to have access to any information, internal computation values or anything at this point of the process. An adversary with access to the table generation phase would trivially recover the key. This event would correspond to a breach of the trusted environment (ex: a supply chain attack) and would therefore be out of the scope of the white-box threat model considered in this project.

Untrusted phase (user)

The green section corresponds to the "user" phase, which represents the deployed application, eventually executed in an untrusted environment. This part actually is the white-box setting, where the application would expose all its inner workings to adversarial observation as stated before.

In this phase, two elements are present:

- **the generated lookup tables:** `tables.h` (generated by the manufacturer but present in the user environment),
- **the encryption binary:** `aes_encrypt` that actively uses these tables to encrypt the plaintext into ciphertext.

The lookup tables are the backbone of the white-box construction: they must encode both the AES-128 algorithm in itself and the secret key used during the encryption process. But most importantly, the tables must do so using a data representation designed to stop the attacker from recovering the manufacturer's key in any way.

The encryption binary is more straightforward: it evaluates the precomputed lookup tables to perform AES-128 encryption, without ever explicitly manipulating the original key. The primary focus of this work is therefore on the construction and protection of these tables (hence file `aes_whitener`). Their usage is derived directly from their design and does not introduce any additional complexity beyond implementation specificity. The distinction between the phases is essential, as all attacks presented later exclusively target the user environment.

Demonstration and validity

To ensure our implementation is correct, we conducted different experiments.

First, during the early development phase, we implemented a classical AES-128 encryption binary (called `aes`). This binary is based on the NIST standard [1], especially Appendix C.1 which specifies reference internal values for each round of the AES computation.

We validated the implementation using the standard test vectors:

```
PLAINTEXT: 00112233445566778899aabbccddeeff
KEY:       000102030405060708090a0b0c0d0e0f
EXPECTED:  69c4e0d86a7b0430d8cdb78070b4c55a
```

The intermediate and final outputs were all compared with the reference values provided in the Appendix cited above to ensure a perfect match, confirming the correctness of the AES-128 encryption binary. Visual confirmation can be seen in Figure 3a where only `m_col` and `k_sch` as internal computation were retained for the sake of brevity.

Figure 3b focuses on the `aes_whitener` and `aes_encrypt` binaries. It highlights how standard AES-128 encryption and the white-box version produce identical outputs, both for the NIST test vectors and for five randomly generated plaintext-key pairs. The testing script `test_wb.sh`, used in the Figure to automate tests and verifications and the main functions used to generate the header `tables.h` are respectively provided in Appendices B and C.

```

> ./aes 000102030405060708090a0b0c0d0e0f 00112233445566778899aabbccddeeff
PLAINTEXT: 00112233445566778899aabbccddeeff
KEY: 000102030405060708090a0b0c0d0e0f

CIPHER (ENCRYPT):
round[ 0].input 00112233445566778899aabbccddeeff
round[ 0].k_sch 000102030405060708090a0b0c0d0e0f
round[ 1].m_col 5f72641557f5bc92f7be3b291db9f91a
round[ 1].k_sch d6aa74fdd2a72fadaa678f1d6ab76fe
round[ 2].m_col ff87968431d86a51645151fa773ad009
round[ 2].k_sch b692cf0b643dbdf1be9bc5006830b3fe
round[ 3].m_col 4c9c1e66f771f0762c3f868e534df256
round[ 3].k_sch b6ff744ed2c2c9bf6c590cbf0469bf41
round[ 4].m_col 6385b79ffc538df997be478e7547d691
round[ 4].k_sch 47f7f7bc93535e03f96c32bcfd058dfdc
round[ 5].m_col f4bcd45432e554d075f1d6c51dd03b3c
round[ 5].k_sch 3caaa3e8a99f9de50f3af57adf622aa
round[ 6].m_col 9816ee7400f87f556b2c049c8e5ad036
round[ 6].k_sch 5e390f7df7a69296a7553dc10aa31f6b
round[ 7].m_col c57e1c159a9bd286f05f4be098c63439
round[ 7].k_sch 14f9701ae35fe28c440adf4d4ea9c026
round[ 8].m_col baa03de7a1f9b56ed5512cba5f414d23
round[ 8].k_sch 47483735a41c65b9e016baf4aebf7ad2
round[ 9].m_col e9f74ecc023828f61bf2ccf2353c21c7
round[10].k_sch 13111d7fe3944a17f307a78bd4d2b30c5
round[10].output 69c4e0d86a7b0430d8cdb7807b04c55a

```

(a) AES encryption internal values, to check with NIST FIPS-197, Appendix C.1

```

> ./test_wb.sh 000102030405060708090a0b0c0d0e0f 00112233445566778899aabbccddeeff
* Cleaning and building base binaries
* Testing key: 000102030405060708090a0b0c0d0e0f
* Testing plaintext: 00112233445566778899aabbccddeeff
[OK] Output match: 69c4e0d86a7b0430d8cdb7807b04c55a
> ./test_wb.sh -random
* Cleaning and building base binaries
--- Test 1 / 5 ---
* Testing key: 5ef07d6544a140635cdfbb89948ac05
* Testing plaintext: b383c558ea110641020ec233f6f108ff
[OK] Output match: fa0c998c0038165b4d22559743e61618
--- Test 2 / 5 ---
* Testing key: efbfa6e170d1c685701d2d6f0f30bbd3
* Testing plaintext: e1a759da35c8a0fa448e77456a0886ae
[OK] Output match: 3e0bd4f8e122b487cd21ef516b712bf6
--- Test 3 / 5 ---
* Testing key: 96f92c52e4584ae4f677d67f58050002
* Testing plaintext: e06a4f87d23c163715980ac8a0f4da05
[OK] Output match: 7ed0fbd16348a57ee5d3aaa379096676
--- Test 4 / 5 ---
* Testing key: 7376c304641533f510aad70a99da084
* Testing plaintext: e7f74e621382eba85da854c0e8581fc2
[OK] Output match: da4ba81e4750ee5590e76b9f5734418c
--- Test 5 / 5 ---
* Testing key: 9567c52f90da49ae91bc8a167553510c
* Testing plaintext: 30d3d7291b3b88845770e222e3a10ae4
[OK] Output match: 3a35bf0d07965fd0145fc09cc0a34ba8
Successfully passed 5 random tests!

```

(b) Practical verifications of white-box implementation correctness

Figure 3: Validation of AES and white-box AES implementations

2.2 A quick reminder about AES

Before moving to the white box implementation, it is necessary to recall the structure and specific properties of AES. AES (Advanced Encryption Standard) is a symmetric encryption algorithm designed by cryptographers Joan Daemen and Vincent Rijmen that was established in 2001 by the NIST (National Institute of Standards and Technology) as the "standardized symmetric encryption algorithm" [11]. It works on 128-bit blocks with keys of length 128, 192 or 256 bits, only the former will be of interest here for the sake of simplicity.

During the computations of the cipher, processed data is arranged as a 4×4 bytes matrix called a **state**. The process of encrypting a block of data consists of 10 rounds (12 and 14 for 192 and 256 bit keys, respectively) by applying the following transformations to the state for each round :

- **AddRoundKey**: XOR the round key derived from the original key (cf. [Key Scheduler](#)) to the current **state**
- **SubBytes**: non-linear byte-wise substitution using the Rijndael S-Box
- **ShiftRows**: cyclic permutation of rows within the **state**
- **MixColumns**: linear transformations applied to columns over $GF(2^8)$

The flowchart of the whole encryption process can be seen in the Figure 4. Note that the final round omits the **MixColumns** transformation.

Key Scheduler:

AES uses a key expansion algorithm to generate a unique key for each round from the original secret key. For AES-128, the key derivation creates 11 round keys of 128 bits each, one for every round including the original key used for the very first **AddRoundKey**.

The key scheduler combines linear and non-linear transformations on 32-bit words and ensures, using the S-Box, word rotation and additions with round-specific constants, that each round key depends on all parts of the original key without introducing relationship between produced keys. The use of a key scheduler does not inherently make brute-force attacks harder but makes structural analysis and attacks more tedious.

This description is intentionally succinct, rather than detailing each transformation, we emphasize their properties: **SubBytes** provides non-linearity, **ShiftRows** and **MixColumns** ensure maximum diffusion, and **AddRoundKey** injects key-dependent bytes into **state**. Those properties are essential components to note during the white-box transformations that we'll introduce later.

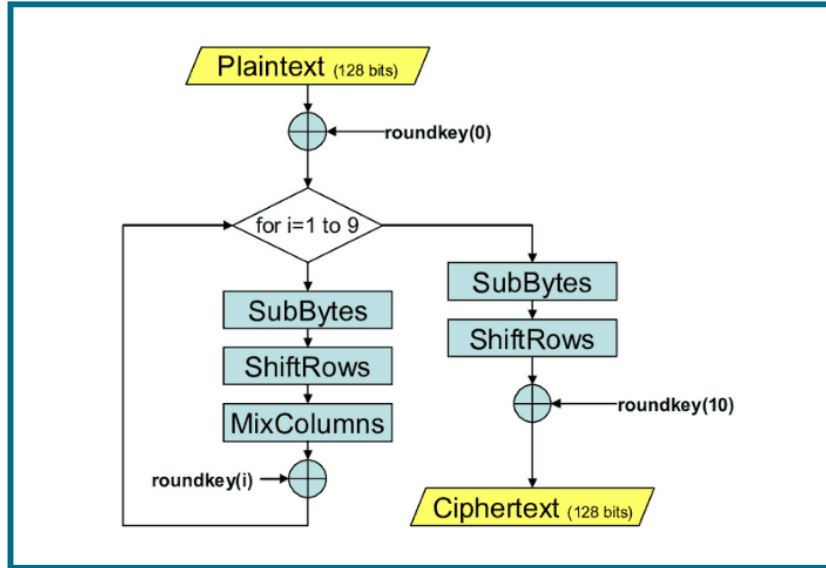


Figure 4: AES encryption flowchart (source: <https://privacycanada.net>)

2.3 Table-based implementation

2.3.1 Changing order of operations

In order to create our table-based implementation of AES-128, the first step is to rearrange the order of operations in the algorithm. That way, we can detect patterns that we can combine, hopefully simplifying the construction of our lookup tables:

1. The first transformation $\text{AddRoundKey}(\text{state}, k_0)$ can be brought inside the `for` loop and $\text{AddRoundKey}(\text{state}, k_9)$ can be pushed out.
2. As `SubBytes` applies a fixed S-Box to the current `state` independently on each byte, it does not matter if we apply it before or after `ShiftRows`, which only permutes the byte positions.
3. `ShiftRows` is immediately followed by `AddRoundKey` with shifted round keys to match the now shifted `state`,

The newly obtained order of operations can be seen below in Figure 5:

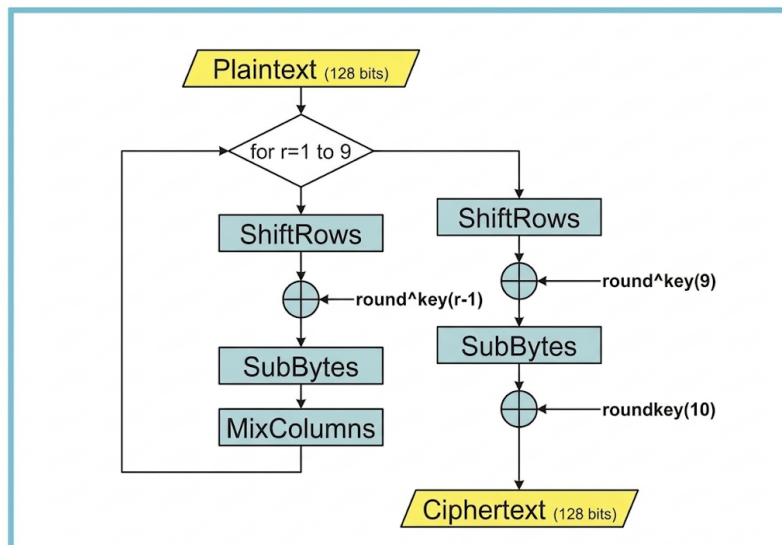


Figure 5: Reordered AES encryption flowchart (Made with Gemini Nano Banana 2)

Now that we have this new ordering, we can see that combining some of the operations is finally possible. Before diving into the specific details, here is a quick rundown of the table constructions:

First, we can combine the `AddRoundKey` and `SubBytes` operations into a single lookup table, pre-computing all the possible values. Secondly, we can pre-compute all values for the `MixColumns` operations as well, creating another lookup table. Furthermore, we'll pre-compute all possible XOR operations to combine the results of the `MixColumns` operations into the next round state, and for obfuscation reasons that will be explained later. Lastly, we'll be able to combine those two tables into one using table composition, achieving a final table that we can recover the final cipher from, with the exception of the final round [10].

With this construction in the trusted phase, we keep the correctness of the original AES algorithm while avoiding the explicit exposition of the key during the encryption phase, as it is baked in the lookup tables that are written in the header file.

2.3.2 Introducing T-boxes

As explained before, we can combine the `AddRoundKey` and `SubBytes` operations into a single lookup table, similarly to the code snippet in Listing 1, which we'll call T-boxes. Mathematically, for a given round r and a specific byte position i within the AES state, let x be the input byte and k_i^r be the corresponding 8-bit round key, where S represents the `SubBytes` operation. We define the T-box lookup table as:

For rounds 1 to 9:

$$T_i^r(x) = S(x \oplus \hat{k}_i^{r-1})$$

For round 10:

$$T_i^{10}(x) = S(x \oplus \hat{k}_i^9 \oplus k_i^{10})$$

Here, $\hat{k}_i^r$ indicates the round key byte after applying the `ShiftRows` transformation. This is necessary as the state has already been shifted at the start of the round. The only exception is round 10 where there is no need for a shifted key.

```

1 void InitTBoxes(uint8_t keys[NB_ROUNDS+1][INPUT_SIZE]) {
2     uint8_t shiftedKey[INPUT_SIZE];
3     /* Rounds 1 to 9 */
4     for (int r = 1; r <= NB_ROUNDS-1; r++) {
5         memcpy(shiftedKey, keys[r-1], INPUT_SIZE);
6         ShiftRows(shiftedKey);
7
8         for (int i = 0; i < INPUT_SIZE; i++) {
9             for (int x = 0; x < 256; x++)
10                TBoxes[r-1][i][x] = SBox[x ^ shiftedKey[i]];
11         }
12     }
13     /* Round 10 */
14     memcpy(shiftedKey, keys[NB_ROUNDS-1], 16);
15     ShiftRows(shiftedKey);
16
17     for (int i = 0; i < INPUT_SIZE; i++) {
18         for (int x = 0; x < 256; x++)
19            TBoxes[NB_ROUNDS-1][i][x] = SBox[x ^ shiftedKey[i]] ^ keys[
20                NB_ROUNDS][i];
21     }
22 }
```

Listing 1: Initialization of T-boxes

Each T-box maps an 8-bit input to an 8-bit output, requiring exactly $2^8 = 256$ entries. For AES-128, which consists of 10 rounds and a 16-byte state, the amount of memory required for the T-boxes is:

$$10 \text{ rounds} \times 16 \text{ bytes/round} \times 256 \text{ entries/table} = 40,960 \text{ bytes (40 KB)}$$

2.3.3 T_{y_i} tables

In standard AES, the `MixColumns` transformation operates on the state matrix column by column, applying a linear transformation through multiplication with a constant 4×4 matrix over the Galois Field $GF(2^8)$. To eliminate the need for on-the-fly finite field arithmetic during the encryption phase, we pre-compute the multiplication of all possible byte values with the coefficients of this constant matrix, creating a set of lookup tables denoted as T_{y_i} boxes [10].

Let MC represent the `MixColumns` constant matrix, where $C_{j,i}$ is the coefficient at row j and column i . For any given 8-bit input byte z , we define the lookup table T_{y_i} to output a 32-bit word corresponding to the scalar multiplication of z by the i -th column of C :

$$T_{y_i}(x) = \begin{pmatrix} MC_{0,i} \bullet x \\ MC_{1,i} \bullet x \\ MC_{2,i} \bullet x \\ MC_{3,i} \bullet x \end{pmatrix}$$

By restructuring the operation in this manner, as implemented in Listing 2, the linear transformation of an entire state column is only four simple memory lookups followed by 3 32-bit XOR additions as follows:

Let $(x_0, x_1, x_2, x_3)^T$ denote the four input bytes of a given state column. The resulting 32-bit column vector Y is computed by evaluating the four corresponding tables and combining their outputs:

$$Y = T_{y_0}(x_0) \oplus T_{y_1}(x_1) \oplus T_{y_2}(x_2) \oplus T_{y_3}(x_3)$$

```

1 void init_TY_TABLES(void) {
2     for(int ty = 0; ty < 4; ty++)
3         for (int copy = 0; copy < 36; copy++)
4             for(int x = 0; x < 256; x++)
5                 TyBoxes[ty][copy][x] = MatrixMult(ty, x);
6 }
```

Listing 2: Initialization of T_{y_i} boxes

Because each T_{y_i} table maps an 8-bit input to a 32-bit (4-byte) output, a single table requires exactly $2^8 = 256$ entries. Furthermore, since the standard `MixColumns` matrix is composed of exactly 4 distinct columns, we only require 4 separate tables to cover all necessary field multiplications.

2.3.4 XOR Tables

Summing the outputs of the four T_{y_i} tables completes the `MixColumns` operation. While we could simply use standard XOR instructions to evaluate this addition, we deliberately replace the operation with pre-computed lookup tables. We do this specifically to prepare the architecture for obfuscation down the line.

Evaluating a full 32-bit addition in a single lookup table would require an impossibly large table composed of 2^{64} entries. To make this practical, the operation is partitioned at the bit level. A single 1-bit XOR table takes two boolean inputs, requiring only $2^2 = 4$ entries.

To sum four state columns, our implementation (see Listing 3) utilizes a binary tree of 3 additions per column, totalling 12 additions per round. Because each addition resolves a 32-bit word, it requires 32 parallel bit-level tables. Across the 9 rounds that execute `MixColumns`, the network generates exactly 3,456 independent tables (9 rounds $\times$ 12 additions $\times$ 32 bits).

```

1 void init_XOR_TABLES(void) {
2     for (int r = 0; r < NB_ROUNDS-1; r++)
3         for (int xi = 0; xi < 12; xi++)
4             for (int n = 0; n < 32; n++)
5                 for (int x = 0; x < 2; x++)
6                     for (int y = 0; y < 2; y++)
7                         XorTables[r][xi][n][x][y] = x ^ y;
8 }
```

Listing 3: Initialization of XOR tables

Since each table contains 4 entries storing a 32-bit word for each single bit (4 bytes) for performance reasons, the total memory footprint for the entire XOR network is precisely 55,296 bytes (approximately 54 KB).

2.3.5 Combining tables

As outlined in Muir’s tutorial on white-box AES, we can further optimize the implementation and obscure the boundary between operations by composing the T-boxes and the T_{y_i} tables into a single lookup table [10]. We’ll refer to these as CT_y tables.

This composition allows the architecture to resolve **SubBytes**, **AddRoundKey** and **MixColumns** operations through a single memory access. Rather than picking the T-box output and feeding it into the T_{y_i} table at runtime, we pre-compute it similarly to previous tables.

For a given round r ($1 \leq r \leq 9$) and a specific byte position i ($0 \leq i \leq 15$) within the AES state, let c be the corresponding column index of that byte in the **MixColumns** matrix ($c = i \bmod 4$). We define the lookup table $CT_{y,i}^r$ as:

$$CT_{y,i}^r(x) = T_{y_c}(T_i^r(x))$$

By restructuring the operation in this manner, as implemented in Listing 4, the evaluation of an entire state column is reduced to four CT_y memory lookups. The resulting 32-bit outputs are then summed using the pre-computed XOR network previously described:

$$Y = CT_{y,0}^r(x_0) \oplus CT_{y,1}^r(x_1) \oplus CT_{y,2}^r(x_2) \oplus CT_{y,3}^r(x_3)$$

```

1 void init_CTY_TABLES() {
2     for (int r = 0; r < NB_ROUNDS - 1; r++) {
3         for (int y = 0; y < 4; y++) {
4             for (int c = 0; c < 4; c++) {
5                 int ty_index = r * 4 + c;
6
7                 for (int i = 0; i < KEY_SIZE; i++) {
8                     for (int x = 0; x < 256; x++) {
9                         uint8_t t = TBoxes[r][i][x];
10                        CTyBoxes[r][y][c][i][x] = TyBoxes[y][ty_index][t];
11                    }
12                }
13            }
14        }
15    }
16 }

```

Listing 4: Initialization of CT_y tables, full code available in Appendix C

Because the **MixColumns** operation is omitted in the final round of AES, this CT_y construction is only applied to rounds 1 through 9 as round 10 relies strictly on standard 8-bit T-boxes. Each CT_y table maps an 8-bit input to a 32-bit (4-byte) output, requiring exactly $2^8 = 256$ entries.

With 16 distinct byte positions per round, the total memory required to store the complete CT_y network is:

$$9 \text{ rounds} \times 16 \text{ tables/round} \times 256 \text{ entries/table} \times 4 \text{ bytes/entry} = 147,456 \text{ bytes (144 KB)}$$

Now that we have our new shiny CTY table and the round 10 T-box, all the **AddRoundKey**, **SubBytes** and **MixColumns** operations can be replaced in our AES algorithm, see Figure 6 for the modified flowchart.

Considering that we’ll only need the combined CT_y tables, the XOR tables and the round 10 T-box for runtime encryption, our total memory footprint comes to:

$$CT_y + T_{XOR} + T_{box10} = 144 + 54 + 4 = 202\text{KB}$$

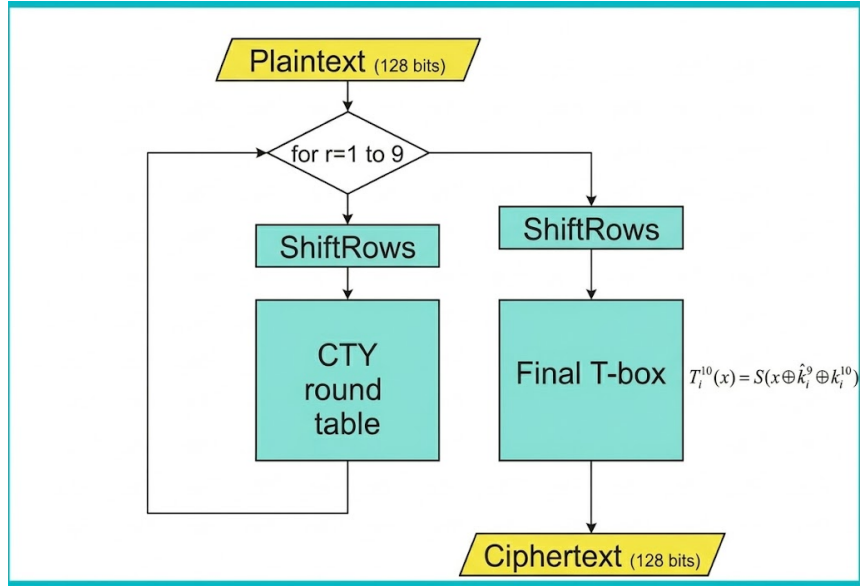


Figure 6: AES flowchart with tables (Made with Gemini Nano Banana 2)

2.4 White-box protection

Right now, our table-based implementation is not protected at all. While the standard key schedule is hidden within the CT_y tables and final T-boxes, the intermediate values flowing between these tables are not obfuscated. As seen in the architecture, the 32-bit column variables and the bit-wise inputs to the XOR network hold exact, unmasked mathematical relationships to the standard AES state.

Because of this, an adversary does not need to reverse-engineer the tables. They can simply observe the memory accesses, intermediate values or the header file. To secure this architecture against such observational and structural cryptanalysis, we must introduce mathematical randomization to the data pathways.

We have to find a way to avoid or complicate an adversary's analysis. This is achieved by wrapping the standard operations in the following layers of protection:

2.4.1 Internal encoding: masking

To prevent the exact values of the state variables from being exposed in memory, we apply boolean masking. Specifically, what is being masked are the intermediate 32-bit column vectors and their bits as they flow between the lookup tables. Instead of processing a true intermediate bit x , the network processes a masked bit $x' = x \oplus m$, where m is a randomly generated boolean value.

This masking strategy is only viable because the XOR operation is strictly linear. When the architecture adds two masked bits together, this linearity ensures that the underlying true values are recoverable:

$$(a \oplus m_a) \oplus (b \oplus m_b) = (a \oplus b) \oplus (m_a \oplus m_b)$$

To understand how this operates continuously across the entire cipher, it is essential to distinguish between the generation phase (`aes.whitener.c`) and the execution phase (`aes.encrypt.c`). The random masks are static and baked into the lookup tables during the construction of the headers, as no randomness is generated during execution.

In standard AES, the `MixColumns` phase sums four 32-bit column vectors using exactly three binary additions. In our architecture, this is evaluated across 32 parallel bit-lanes. Let v_0, v_1, v_2 , and v_3 represent the theoretical unmasked bits of a single lane originating from the four CT_y tables. The architecture sums these using a binary tree structure:

- **Node 0 (Left Branch):** $left = \text{XOR}(v_0, v_1)$
- **Node 1 (Right Branch):** $right = \text{XOR}(v_2, v_3)$
- **Node 2 (Accumulator):** $acc = \text{XOR}(left, right)$

During execution, the runtime evaluates this pathway across the parallel bit-lanes utilizing only lookup tables, entirely bypassing arithmetic operations (see Listing 5).

```

1  /* 1. Fetch the 32-bit masked vectors from the CTY tables */
2  uint32_t v0 = CTY_TABLES[r][0][c][state[...]];
3  uint32_t v1 = CTY_TABLES[r][1][c][state[...]];
4  uint32_t v2 = CTY_TABLES[r][2][c][state[...]];
5  uint32_t v3 = CTY_TABLES[r][3][c][state[...]];
6
7  /* 2. Evaluate the binary tree topology */
8  uint32_t left  = XOR32(v0, v1, r, x0); /* Node 0 (Left Branch) */
9  uint32_t right = XOR32(v2, v3, r, x1); /* Node 1 (Right Branch) */
10 uint32_t acc   = XOR32(left, right, r, x2); /* Node 2 (Accumulator) */
11
12 /* Processing the bit lanes */
13 for (int bit = 0; bit < NB_XOR_BITS; bit++) {
14     uint8_t ba = (a >> (31 - bit)) & 1U;
15     uint8_t bb = (b >> (31 - bit)) & 1U;
16     /* Fetch precomputed bit contribution and accumulate */
17     res ^= XOR_BIT_TABLES[round][xor_index][bit][ba][bb];
18 }

```

Listing 5: Runtime execution of the XOR tree for a single column

During generation, the CT_y tables are constructed to output bits protected by independent random masks: m_0, m_1, m_2 , and m_3 . Therefore, the execution phase only ever handles $v'_i = v_i \oplus m_i$.

To process these securely, the whitener generates a specific 4-entry lookup table for each XOR node. These tables are mathematically constructed to internally cancel the incoming masks from the previous layer and apply a newly generated intermediate mask.

For the left branch (**Node 0**), the whitener assigns a random intermediate mask m_{left} . The table T_{Node0} is explicitly built to absorb m_0 and m_1 :

$$T_{Node0}(v'_0, v'_1) = v'_0 \oplus v'_1 \oplus (m_0 \oplus m_1 \oplus m_{left})$$

Algorithmically, the whitener achieves this by calculating the mathematical difference between the incoming and outgoing masks, baking this Δ directly into the pre-computed bit tables (Listing 6).

```

1  /* Calculate delta: m_a XOR m_b XOR m_out */
2  uint8_t delta = mask_a ^ mask_b ^ mask_out;
3
4  /* Iterate over the 4 possible masked bit combinations (00, 01, 10, 11) */
5  for (int a = 0; a < 2; a++) {
6      for (int b = 0; b < 2; b++) {
7          /* Compute the true XOR and apply the mask delta simultaneously */
8          uint8_t out_bit = (uint8_t)(a ^ b ^ delta);
9
10         /* Store the resulting bit shifted to its required destination */
11         XOR_BIT_TABLES[r][xi][bit_index][a][b] = out_bit;
12     }
13 }

```

Listing 6: Generation phase compiling the masked XOR nodes

During execution, the inputs corresponding to the left branch compute as follows:

$$(v_0 \oplus m_0) \oplus (v_1 \oplus m_1) \oplus (m_0 \oplus m_1 \oplus m_{left}) = (v_0 \oplus v_1) \oplus m_{left} = left'$$

The right branch (**Node 1**) operates identically, absorbing m_2 and m_3 to produce $right'$ protected by m_{right} .

Finally, the Accumulator (**Node 2**) must link these branches. The whitener sets a final output mask for this round, m_{acc} , and constructs the table to expect the intermediate masks:

$$T_{Node2}(left', right') = left' \oplus right' \oplus (m_{left} \oplus m_{right} \oplus m_{acc}) = acc'$$

The masking network must not only chain within a single round but also seamlessly bridge the transition between consecutive rounds. At the conclusion of round $r - 1$, the XOR tree outputs a fully masked 32-bit state. Through the implicit routing of the `ShiftRows` operation, a specific masked bit and its corresponding mask m_{acc}^{r-1} are physically shifted to a new column position to serve as the input for round r .

Therefore, the masked state bit arriving at the input of round r is mathematically defined by its origin in the previous round:

$$x' = x \oplus m_{acc}^{r-1}$$

For the cipher to remain mathematically correct, the CT_y table in round r must be explicitly built to expect and cancel this exact incoming mask. During the generation phase, the whitener constructs the encoded table CT_y^r by baking the cancellation of m_{acc}^{r-1} directly into the table's logic, while simultaneously applying a new set of output masks (m_{out}^r) for the current round's XOR tree:

$$CT_y^r(x') = CT_y^r(x' \oplus m_{acc}^{r-1}) \oplus m_{out}^r$$

When the execution phase evaluates this table with the incoming masked bit $x' = x \oplus m_{acc}^{r-1}$, the input mask perfectly cancels out inside the table index.

Listing 7 illustrates how the generation phase retrieves this specific mask from the previous round and hardcodes the cancellation into the T_{BOX} construction.

```

1  /* 1. Retrieve the output mask from the previous round's Accumulator (Node
2     2) */
3  uint8_t mask_hi = M_XOR[prev_r][prev_final_xor][prev_row * 2];
4  uint8_t mask_lo = M_XOR[prev_r][prev_final_xor][prev_row * 2 + 1];
5  uint8_t input_mask = (mask_hi << 4) | mask_lo; /* m_{acc}^{r-1} */
6
7  /* 2. Bake the cancellation into the table for all 256 possible bytes */
8  for (int x = 0; x < ALL_BYTES_FOR_KEY; x++) {
9      /* x represents the masked incoming data. Strip it to find the true
10         value. */
11      uint8_t real_x = x ^ input_mask;
12
13      /* Evaluate standard AES operations on the true, unmasked value */
14      T_BOXES[r-1][i][x] = SBox[real_x ^ shiftedKey[i]];
15  }
16  /* Note: M_CTY (the output mask) is applied subsequently during
17     init_CTY_TABLES */

```

Listing 7: Compiling the T-Box to absorb the previous round's mask

Consequently, the algebraic expansion of this table evaluation proves that the incoming mask is perfectly absorbed:

$$\begin{aligned}
 CT_y^r(x') &= CT_y^r((x \oplus m_{acc}^{r-1}) \oplus m_{acc}^{r-1}) \oplus m_{out}^r \\
 &= CT_y^r(x) \oplus m_{out}^r
 \end{aligned}$$

The standard AES operations (`SubBytes`, `AddRoundKey`, `MixColumns`) are mathematically applied to the true, unmasked value x , and the result is immediately output with the new mask m_{out}^r .

It is important to note that round 1 represents an edge case in our architecture. Because there is no preceding XOR tree to supply an internal mask, the white-box accepts standard, raw plaintext. The initial CT_y^1 tables simply evaluate the unmasked plaintext x and immediately apply the first internal mask m_{out}^1 to initiate the protected pipeline:

$$CT_y^1(x) = CT_y^1(x) \oplus m_{out}^1$$

Conversely, the final round ($r = 10$) omits the `MixColumns` operation, meaning there is no final XOR tree. The CT_y tables are replaced by the final T-box that executes only the `SubBytes` and `AddRoundKey` steps, expecting the final output mask from the round 9 CT_y table.

Because the implementation outputs clear ciphertext, these final tables do not generate a new internal mask. The whitener generates the final table, T_{final} , such that it cancels the internal mask and outputs the raw ciphertext block:

$$T_{final}(x') = S(x' \oplus m_{acc}^9 \oplus \hat{k}_i^9) \oplus k_i^{10}$$

Through this chained pre-computation across the XOR circuit, the unmasked values never explicitly show up in memory. The architecture continuously mutates the data from one masked state directly into another, ensuring that static memory analysis reveals only randomly masked elements, rendering extraction difficult. Internal encodings applied to the proof-of-concept can be seen in Appendix C.

While masking helps us cover the semantic functions of our CTY table by applying random masks, it doesn't protect the XOR tables from static analysis at all. Knowing that the XOR tables store a single bit XOR operation result, the mask can be guessed easily by looking at the table. With that, the masks can be uncovered, rendering the semantic security of our architecture void.

2.4.2 Mixing bijections

To mitigate this vulnerability, we must prevent the attacker from analyzing these single-bit XOR tables in isolation, as they render tracing a single bit from the CT_y tables through the XOR tree to extract its mask trivial.

To disrupt this structural simplicity, we introduce linear mixing bijections in the form of randomized 32-bit permutations. While these permutations are linear operations and do not provide absolute immunity, they provide crucial spatial diffusion that severely complicates static routing analysis.

During the initialization phase, the whitener generates a unique, random 32-bit permutation array (MIX_POS) for each of the four columns in every round.

When constructing the CT_y tables, this mixing bijection is applied to the 32-bit output of the MixColumns operation *before* the boolean mask (M_CTY) is applied. As a result, the 32 bits representing the column state are shuffled across the 32 bit-lanes.

During the execution phase, the first two layers of the binary XOR tree (**Node 0** and **Node 1**) perform their evaluations on these scrambled bit-lanes. An attacker looking at the memory cannot easily determine which logical bit of the AES state corresponds to which bit-lane in the XOR tree.

For the cipher to function correctly, this permutation must be inverted before the data is passed to the next round. Rather than adding a dedicated de-permutation step, our architecture achieves inversion by baking the un-routing directly into the final accumulator node (**Node 2**) of the XOR tree.

As shown in the generation logic, when the whitener compiles the lookup tables for **Node 2**, it maps the permuted source bit (**src**) back to its original, unpermuted destination index (**dst**):

```

1 // Node 2 (Root): route permuted source bit into its unpermuted destination
2 dst = MIX_POS[r][col][src];
3 out_bit_index = dst;
4
5 // ... mask delta calculations ...
6
7 // The 1-bit result is shifted directly into the corrected 'dst' position
8 XOR_BIT_TABLES[r][xi][src][a][b] = out_bit ? (1U << (31 - dst)) : 0U;
```

Listing 8: Baking the inverse permutation into the final XOR node

As the runtime evaluates **Node 2**, it seamlessly calculates the final masked XOR result while inherently restoring the bit to its correct architectural position.

This application of mixing bijections thoroughly obscures the physical routing of the data. Consequently, the adversary is now compelled to first unravel the 32-bit cross-lane permutations for every single column, establishing a substantially higher barrier against static analysis.

3 Hacking the box

Now that our white-box architecture design is established, the next crucial step is verifying its security through empirical testing using known effective attacks on AES. Because the white-box threat model exposes the implementation to a vast variety of attack vectors, exhaustive testing is highly impractical. For this reason, we focused our evaluation on two of the most prominent attacks against AES, namely DFA and DCA.

3.1 Differential Fault Analysis (DFA)

In a white-box context, adversaries not only observe the execution but can also actively alter the runtime environment. Differential Fault Analysis (DFA) is an active side-channel attack that exploits this capability. First introduced for AES by Dusart, Letourneux, and Vivolo in 2003 [8], DFA recovers the secret key by introducing controlled errors into the cryptographic computation and mathematically comparing the resulting faulty ciphertext with the correct reference ciphertext.

The classic DFA attack on AES targets the final rounds of the cipher. If an attacker injects a single-byte fault at the input of the 9th round, the error propagates through the 9th round's `SubBytes`, `ShiftRows`, and `MixColumns`. Crucially, the `MixColumns` operation causes this single-byte fault to diffuse into a full column of four faulty bytes.

In the 10th (and final) round, these four bytes are dispersed across four different columns by the `ShiftRows` operation. After the final `SubBytes` and `AddRoundKey`, the resulting faulty ciphertext differs from the expected correct ciphertext in exactly four specific byte positions. By analysing the `XOR` difference between the correct and faulty ciphertexts, the attacker can formulate a system of equations based on the inverse S-Box and the `MixColumns` differentials. Solving these equations reveals the 10th round key.

To demonstrate this vulnerability, we implemented an automated DFA attack tailored to our white-box architecture. Our testing script injects the fault immediately before the 9th round. It attempts to recover the key in four distinct phases:

- **Collection:** The script encrypts a plaintext to obtain a reference ciphertext. It then injects various `XOR` faults into the 16 bytes of the 9th round input, saving the resulting faulty ciphertexts into a log file.
- **Classification:** Each faulty ciphertext is compared against the reference. Based on the inverse `ShiftRows` function, the differing bytes must fall into one of four predictable groups (e.g., bytes 0, 7, 10, and 13). The script categorizes the traces by these groups.
- **Candidate Intersection:** For each group, the script iterates through all possible values for the corresponding four bytes of the 10th round key. It calculates backwards through the final `AddRoundKey` and `SubBytes` to verify if the state differences align with a valid `MixColumns` output originating from a single-byte fault. Each fault reduces the pool of valid key candidates. By computing the intersection of the candidate sets from just a few faults, the script isolates the single correct key candidate.
- **Key Schedule Reversal:** Once all four column groups are resolved, the complete 128-bit 10th round key is assembled. An inverse key schedule algorithm is then applied to traverse backwards, ultimately recovering the original AES master key.

To automate this attack via a Python script, we needed a reliable way to inject faults. Leveraging our white-box access, we simply modified the C source code to accept fault parameters directly from the command line, outputting the results of such faulty executions to feed our script.

Listing 9 shows how the runtime intercepts the state array at the start of round 9 and applies the fault before the data passes through the `ShiftRows` and `CTy` tables.

```

1  /* loop through rounds 1 to 9 */
2  for (int r = 0; r < NB_ROUNDS-1; r++) {
3      /* single-byte DFA fault injected at the 9th round input */
4      if ((r + 1) == (NB_ROUNDS - 1) && fault_byte >= 0 && fault_byte <
5          KEY_SIZE && fault_xor != 0) {
6          state[fault_byte] ^= fault_xor;
7          printf("fault[r=%d,b=%d,x=0x%02X]\n", r + 1, fault_byte,
8              fault_xor);
9      }
10     ShiftRows(state);
11     uint8_t new_state[KEY_SIZE];
12     for (int c = 0; c < STATE_NB_ROW; c++) {
13         /* process each column */
14         // ... CTY_TABLES and XOR32 evaluations ...

```

Listing 9: Extract of `dfa.c`: Fault injection at the input of the 9th round

To resolve the 10th round key from these faulty ciphertexts during the **Candidate Intersection** phase, the script relies on the properties of the `MixColumns` and `SubBytes` operations.

Let f be the value of the injected single-byte fault. When passing through the 9th round's `MixColumns` operation, this single fault diffuses into a known column difference ΔY . For example, if the fault is located at the first byte of a column, the resulting difference over $\text{GF}(2^8)$ is calculated as:

$$\Delta Y = \text{MixColumns} \begin{pmatrix} f \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} \Delta Y_0 \\ \Delta Y_1 \\ \Delta Y_2 \\ \Delta Y_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} f \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 02 \cdot f \\ 01 \cdot f \\ 01 \cdot f \\ 03 \cdot f \end{pmatrix}$$

Let C and C^* denote the correct and faulty ciphertexts respectively. By reverting the final `AddRoundKey` and `SubBytes` operations, the difference before the 10th round ΔY_i can be expressed as a function of the targeted 10th round key byte $K_{10,i}$:

$$\Delta Y_i = S^{-1}(C_i \oplus K_{10,i}) \oplus S^{-1}(C_i^* \oplus K_{10,i})$$

where S^{-1} represents the Inverse S-Box.

Combining this relation with the `MixColumns` differentials derived above, we obtain a strict system of equations for the four affected bytes:

$$\begin{aligned} S^{-1}(C_0 \oplus K_{10,0}) \oplus S^{-1}(C_0^* \oplus K_{10,0}) &= 02 \cdot f \\ S^{-1}(C_1 \oplus K_{10,1}) \oplus S^{-1}(C_1^* \oplus K_{10,1}) &= 01 \cdot f \\ S^{-1}(C_2 \oplus K_{10,2}) \oplus S^{-1}(C_2^* \oplus K_{10,2}) &= 01 \cdot f \\ S^{-1}(C_3 \oplus K_{10,3}) \oplus S^{-1}(C_3^* \oplus K_{10,3}) &= 03 \cdot f \end{aligned}$$

For each of the four bytes in a group, the adversary iterates through all 256 possible guesses for the key byte $K_{10,i}$ and computes the corresponding fault value f . A key hypothesis is only considered valid if all four equations yield the exact same fault value f . Each injected fault significantly reduces the amount of valid candidates. In practice, only a small number of faults is required to isolate the valid key bytes, as illustrated in Figure 7, which shows the exponential reduction of the key candidate space.

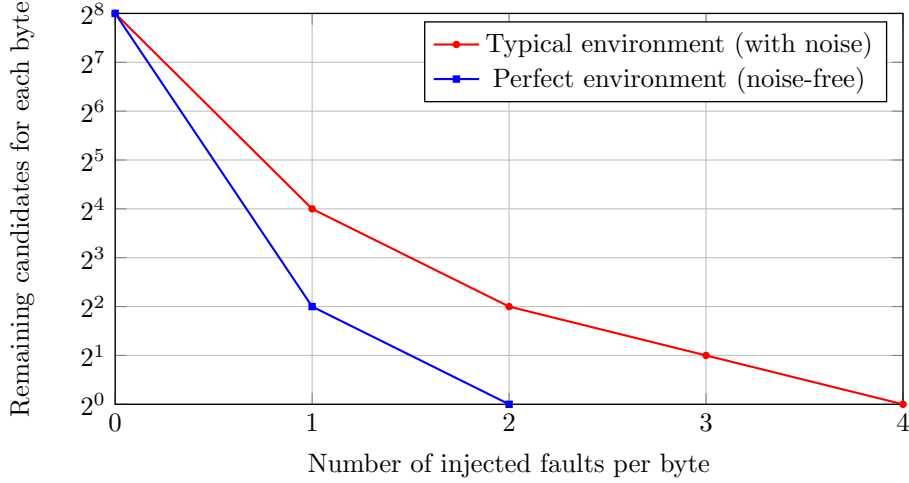


Figure 7: Conceptual reduction of AES key byte candidates per errors injected

Once the key candidate pool is drastically reduced, the script successfully recovers the corresponding key byte. Repeating this process for all bytes groups finally yields the full round key.

Our experiments confirm that the white-box implementation is vulnerable to Differential Fault Analysis. The full 128-bit secret key can be consistently recovered with only a small number of fault injections, typically between 2 and 4 per byte [15].

It should be noted that Figure 7 represents an idealized model of the candidate reduction process based on this typical behaviour. However, in our proof of concept setting, fault injections are perfectly controlled and no measurement noise is present. This causes the attack to be even more efficient in practice (see blue plot just above), only 1 to 2 faults being generally enough to retrieve the secret key.

The success of this attack highlights a critical limitation in our white-box design: while boolean masking and mixing bijections effectively hide the intermediate state from observation, they provide no protection against fault injection.

The attack works perfectly because our obfuscation methods are designed to be functionally equivalent to standard AES. When a fault is injected into the masked state, the error propagates through the masked `MixColumns` and `SubBytes` operations exactly as it would in an entirely unprotected black-box implementation. The random masks and bijections merely scramble the representation of the data in memory as they do not alter the underlying mathematical relationship between the fault, the key, and the final ciphertext.

3.2 Differential Computation Analysis (DCA)

In a white-box model, the adversary can observe the entire execution of the algorithm. In particular, they can observe, log and store internal computations: intermediate values used during the encryption process.

In a perfect white-box implementation, all internal values should be akin to random data and reveal no meaningful information. However, in practice, implementations often fail to fully achieve this property. An attacker can therefore attempt to detect statistical dependencies between internal values and key-dependent computations.

This observation led to the development of Differential Computation Analysis (DCA), introduced in 2016 by Bos et al., which provides a methodology to detect and exploit such correlations. It is an analogous version of the Differential Power Analysis (DPA), a side-channel attack that studies the power consumption of a hardware device to infer internal computation values [5].

To perform a DCA attack on AES-128, the adversary,

- collects a large number of execution traces for different plaintexts,
- selects an internal variable (like a byte in memory),
- makes a hypothesis on a small part of the key ,
- computes a predicted value,
- measures the statistical correlation between the prediction and the observed traces,
- finally, if the hypothesis is correct, the prediction should correlate with previously gathered data, otherwise it would just behave like random noise
- with all our validated hypothesis in hand, we obtain a round key that we can usually decode into the original key, breaking the white-box

Let's now see how the attack was implemented to break our white-box:

First of all, the first non-trivial step of our white-box encryption is a lookup within the CTY_TABLES. Each lookup takes one byte as input and gives out 32 bits as an output (see below).

```

21      /* fetch masked 32-bit result for Row 0, 1, 2 & 3 */
22      uint32_t v0 = CTY_TABLES[r][0][c][state[c*STATE_NB_ROW + 0]];
23      uint32_t v1 = CTY_TABLES[r][1][c][state[c*STATE_NB_ROW + 1]];
24      uint32_t v2 = CTY_TABLES[r][2][c][state[c*STATE_NB_ROW + 2]];
25      uint32_t v3 = CTY_TABLES[r][3][c][state[c*STATE_NB_ROW + 3]];
26
27      /* trace collection for DCA attack */
28      if (!quiet && r == 0) {
29          if ( && c == 0) printf("TRACE_");
30          printf("%08x%08x%08x%08x", v0, v1, v2, v3);
31          if (c == STATE_NB_ROW - 1) printf("\n");
32      }

```

Listing 10: Extract of AES_whitener.c: CTY_TABLES application and trace collection

As these tables merge three AES operations (AddRoundKey, SubBytes and MixColumns), each output can be written as follows:

$$\begin{aligned}
 T[x] &= \text{MixColumns}(\text{SubBytes}(\text{AddRoundKey}(x))) \\
 T[x] &= \text{MixColumns}(\text{SBox}(x \oplus k)),
 \end{aligned}$$

We can see a possible correlation between the output of the lookup table $T[x]$, the key byte k and the input x .

Even if this dependency is obfuscated with our white-box methods like masks and random bijections, the underlying algebraic structure of AES is indeed preserved. Due to this property, the output still directly depends on the key byte k and the input x . This is precisely what we are targeting in this DCA attack by extracting the resulting vectors into a file to have a statistical sample (see above). In true white-box attacks, the value is extracted using debuggers but as we have access to the code, it is simpler to extract it directly.

After we gathered the traces, we use a python file (`run_dca.py`) to run the statistical attack that will be described below.

Beforehand, we have to note that the very first round is applied after a `ShiftRows(state)` transformation. This means all bytes shown during the first round are already permuted. We thus need to apply the same permutation to the plaintext before performing the key hypothesis in order to align our predictions to the actual internal state during encryption.

Then, for each key hypothesis byte $k \in \{0, \dots, 255\}$ and for each plaintext byte p_i (after permutation), we compute:

$$z = \text{SBox}(p_i \oplus k),$$

and get a value representing the information leak using the Hamming weight:

$$I = \text{HW}(z).$$

We use the Hamming Weight because the exact nature of the leak is unknown. Here we work on 32-bit values obtained from table lookups without knowing how the leak can emerge within the observed value. The Hamming Weight provides an approximation that stays statistically correlated to the data without relying on intricate computations.

Each collected trace is split into individual bits. For each one, we compute the linear correlation between the predicted leakage and the observed traces. As we do not know in advance which bit leaks the most information, we evaluate all of them for each trace and keep only the biggest correlation value. The computation can be summed up with the following formula:

$$\max_b |\text{corr}(I, T_b)|.$$

where b indicates the bit we're working on and T_b the corresponding trace. The 'correct' key byte is the hypothesis byte maximising this value.

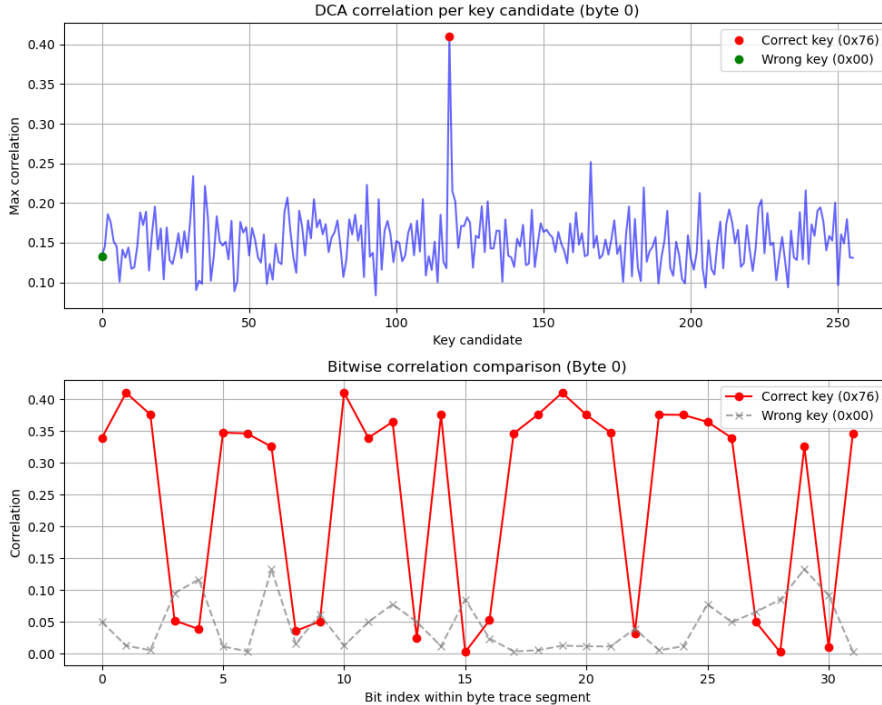


Figure 8: Top: correlation for each key candidate; bottom: bitwise correlation comparison between correct and incorrect key hypothesis

Figure 8 illustrates the statistical distinguisher used in our DCA attack on the first byte of the key. The upper plot shows the maximum correlation score for each key candidate from 0 to 255: one hypothesis stands out by producing a very high correlation value (≈ 0.42), while all others hypotheses remain close to the background noise level (≈ 0.15 on average). The lower graph details the bitwise correlations for the correct key. It highlights that several bits leak significant information, whereas a wrong key exhibits only weak and irregular correlations.

We assume this byte is the 'correct' hypothesis. This process is then done for the remaining 15 bytes until we get a 128-bit key with the highest correlation. As the state was permuted by **ShiftRows** before the first round, we cancel this operation by applying the inverse **ShiftRows** function to the obtained shifted key in order to recover the original one.

Figure 9 evaluates the efficiency of the DCA attack depending on the amount of traces collected. It compares two versions of our implementation: with and without obfuscation. We can observe that both curves are extremely similar: the number of correctly recovered bytes increases rapidly with the number of traces, reaching more than 50% key recovery for about 75 traces and full key recovery for about 250 traces for both versions.

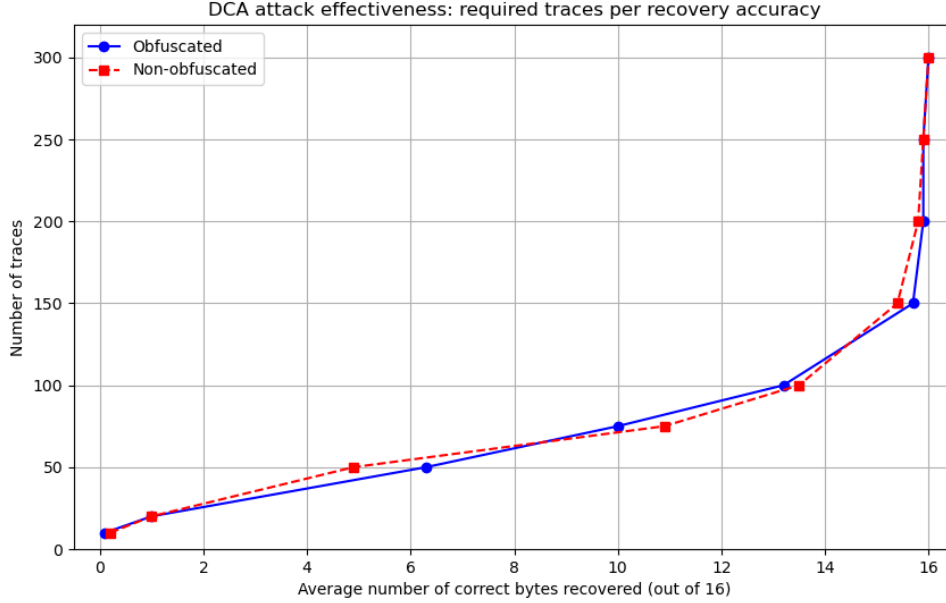


Figure 9: Comparison of correctly recovered bytes per amount of traces for obfuscated and non-obfuscated white-box

With our trace collection methods, acquiring approximately 300 traces takes about 10 seconds on any run-of-the-mill machine, resulting in a total attack time on the order of a minute.

This demonstrates that all applied obfuscation do not sufficiently decorrelate the manipulated data from the key, having no significant impact on the effectiveness of the DCA attack. This result confirms that without proper internal/external encodings, this white-box implementation is extremely vulnerable to statistical cryptanalysis.

4 Conclusion

This report presented the design and implementation of a white-box AES-128 proof of concept. The system successfully reproduces standard AES encryption through the use of precomputed lookup tables, while different methods like masking and arithmetic transformations were introduced to conceal intermediate values and avoid direct key exposure.

However, while the methods used provided a good obfuscation of key-related data, security evaluations confirmed they were insufficient against a white-box threat model. Differential Fault Analysis recovers the secret key by injecting faults and Differential Computation Analysis exploits residual algebraic and statistical leaks in the tables. Both attacks succeed with low effort, often recovering the secret key in mere seconds. This indicates that our protections provide obfuscation instead of effective resistance against practical attacks.

One could argue that a limitation in our implementation is the absence of extreme internal encodings (other than masking) and external encodings as described in Muir’s tutorial that this work is based on. They were not implemented due to their complexity relative to the scope of the project and small expected benefits.

Firstly, these encoding techniques were originally introduced to counter early structural attacks on white-box implementations, like key extraction through table decomposition or algebraic simplifications [10]. Though they are still used in practice as they are still effective against those older attacks, they were not designed to resist modern attacks like side-channels.

In particular, as shown in Section 3, internal encodings do not remove underlying statistical dependencies between manipulated data and key bytes, rendering them virtually useless against side channels attacks like DCA [4].

As for external encodings, they serve a different purpose: they wrap inputs and outputs interface to prevent unwanted use of the encryption implementation as a black-box oracle [10]. In essence, they do not strengthen resistance against white-box adversaries with full visibility of internal computations, and therefore would not mitigate the attacks demonstrated earlier.

Recent research in white-box cryptography shows an attempt to shift from simply hiding the key to explicitly resisting trace and fault-based attacks. Those modern stronger countermeasures include higher-order non-linear encodings, masking of key intermediates and randomized memory accesses to break alignment between traces and internal values at the cost of a heavier memory load. Additionally, active resistance against fault injection has become a primary objective, leading to the emergence of new formal security models in the recent years that aim to stem both trace and fault attack scenarios. [7].

More fundamentally, the presented limitations align with theoretical results on obfuscation. The impossibility Theorem of Barak et al.[3] shows that a general-purpose perfect obfuscator program cannot exist. Although the theorem’s definition might be a bit too broad to exclude the possibility of having a realistically perfect obfuscator for a specific algorithm, it still highlights limits of relying on code transformation alone for security.

Overall, this work illustrates the gap between hiding critical data and protecting it within an untrusted environment. In this sense, Kerckhoffs’s principle is confirmed once again: security can’t rely on the obscurity of the implementation alone.

References

- [1] Advanced encryption standard (aes), November 2001. Federal Information Processing Standards Publication.
- [2] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, and Martin Bjerregaard Jepsen. Analysis of software countermeasures for whitebox encryption. *IACR Transactions on Symmetric Cryptology*, 2017.
- [3] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs (extended abstract). In *Advances in Cryptology – CRYPTO 2001: 21st Annual International Cryptology Conference*, volume 2139 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2001. Full version available at <http://eccc.hpi-web.de/report/2001/057/>.
- [4] E. Alpirez Bock, C. Brzuska, W. Michiels, and A. Treff. On the ineffectiveness of internal encodings – revisiting the dca attack on white-box cryptography. In Bart Preneel and Frederik Vercauteren, editors, *Applied Cryptography and Network Security (ACNS 2018)*, volume 10892 of *Lecture Notes in Computer Science*, Cham, 2018. Springer.
- [5] Joppe W. Bos, Christophe Hubain, Wil Michiels, and Philippe Teuwen. Differential computation analysis: Hiding your white-box designs is not enough. In *Cryptographic Hardware and Embedded Systems (CHES)*, 2016.
- [6] CBC News. The pros, cons, and future of drm. <https://www.cbc.ca/>, August 2009.
- [7] Alex Charlès and Aleksei Udovenko. Haystack ciphers: White-box countermeasures as symmetric encryption. Cryptology ePrint Archive, Paper 2025/1635, 2025.
- [8] Pierre Dusart, Gilles Letourneux, and Olivier Vivolo. Differential fault analysis on a.e.s. *Lecture Notes in Computer Science (ACNS 2003)*, 2846:293–306, 2003.
- [9] Auguste Kerckhoffs. La cryptographie militaire. *Journal des sciences militaires*, 9:5–83, 161–191, 1883.
- [10] James A. Muir. A tutorial on white-box aes. *IACR Cryptol. ePrint Arch.*, 2013:104, 2013.
- [11] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, 2001. Fetched on <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf> Last updated May 9, 2023.
- [12] Statista Research Department. Volume of data or information created, captured, copied, and consumed worldwide from 2010 to 2029. Statista, 2025. Accessed: 2026-04-12.
- [13] Philippe Teuwen and Charles Hubain. Differential fault analysis on white-box AES implementations. Quarkslab’s blog, December 2016.
- [14] Guillaume Vinet. Optimized white-box tracing for efficient side-channel attacks. eShard Blog, August 2024.
- [15] Xusen Wan, Jinbao Zhang, Shi Cheng, Weixiang Wu, and Jiehua Wang. A novel dfa on aes: Based on two-byte fault model with discontinuous rows. *Applied Sciences*, 13(14):8282, 2023.

A AES encryption using white-box lookup tables

```
1 static void AES_128_WB(uint8_t* state, int fault_byte, uint8_t fault_xor,
2 bool quiet) {
3     for (int r = 0; r < NB_ROUNDS-1; r++) { /* loop through rounds 1-9 */
4         if ((r + 1) == (NB_ROUNDS - 1) && fault_byte >= 0 && fault_byte <
5             KEY_SIZE && fault_xor != 0) {
6             state[fault_byte] ^= fault_xor; /* single-byte DFA fault at
7             selected round input */
8             if (!quiet)
9                 printf("fault[r=%d,b=%d,x=0x%02X]\n", r + 1, fault_byte,
10                    fault_xor);
11         }
12         ShiftRows(state); /* shift the rows of the state matrix */
13
14         uint8_t new_state[KEY_SIZE];
15         /* process each column of the state matrix independently */
16         for (int c = 0; c < STATE_NB_ROW; c++) {
17
18             /* compute 1st, 2nd and 3rd XOR node index for this column */
19             const int x0 = 3 * c;
20             const int x1 = x0 + 1;
21             const int x2 = x0 + 2;
22
23             /* pass each masked byte of the column into CTY_TABLES to apply
24             SubBytes, AddRoundKey, and MixColumns */
25
26             /* fetch masked 32-bit result for Row 0, 1, 2 & 3 */
27             uint32_t v0 = CTY_TABLES[r][0][c][state[c*STATE_NB_ROW + 0]];
28             uint32_t v1 = CTY_TABLES[r][1][c][state[c*STATE_NB_ROW + 1]];
29             uint32_t v2 = CTY_TABLES[r][2][c][state[c*STATE_NB_ROW + 2]];
30             uint32_t v3 = CTY_TABLES[r][3][c][state[c*STATE_NB_ROW + 3]];
31
32             /* trace collection for DCA attack */
33             if (!quiet && r == 0) {
34                 if (&& c == 0) printf("TRACE_");
35                 printf("%08x%08x%08x%08x", v0, v1, v2, v3);
36                 if (c == STATE_NB_ROW - 1) printf("\n");
37             }
38
39             /* add Row 0 and Row 1 using Node 0's table */
40             uint32_t left = XOR32(v0, v1, r, x0);
41             /* add Row 2 and Row 3 using Node 1's table */
42             uint32_t right = XOR32(v2, v3, r, x1);
43             /* add left and right using Node 2 to get final column */
44             uint32_t acc = XOR32(left, right, r, x2);
45
46             /* can only be done once acc is computed entirely */
47             for (int i = 0; i < STATE_NB_ROW; i++) /* unpack the 32-bit
48             accumulator back into 4 separate bytes */
49                 new_state[c*STATE_NB_ROW + i] = (acc >> (NB_XOR_NIBBLES * (
50                     STATE_NB_ROW - 1 - i))) & 0xff; /* extract the
51                 appropriate byte */
52         }
53         memcpy(state, new_state, KEY_SIZE); /* update state */
54         if (!quiet) {
55             printf("round[%d].n_col\t", r + 1);
56             print_hex(state, KEY_SIZE);
57         }
58     }
59     ShiftRows(state); /* perform the final ShiftRows for Round 10 */
60     for (int i = 0; i < KEY_SIZE; i++) /* apply final T-Box */
61         state[i] = FINAL_T_BOX[i][state[i]];
62 }
```

Listing 11: AES_128_WB: encryption using implemented white-box version of AES

B Test script to validate white-box implementation

```
1 #!/bin/bash
2 set -e
3
4 # generate a random 16-byte hex string
5 generate_random_hex() { head -c 16 /dev/urandom | xxd -p | tr -d '\n'; }
6
7 # run a single test
8 run_test() {
9     local KEY="$1" PLAINTEXT="$2"
10    echo " * Testing key:      $KEY"
11    echo " * Testing plaintext: $PLAINTEXT"
12
13    # base AES encryption output as reference
14    local AES_OUT=$(./aes "$KEY" "$PLAINTEXT" | tail -n 1 | grep -oE '[0-9a-fA-F]{32}')
15
16    # generating lookup table header for this specific key
17    ./aes_whitener "$KEY" > /dev/null
18
19    # rebuilding white-box binary
20    rm -f aes_encrypt; make aes_encrypt > /dev/null
21
22    # running white-box binary
23    local WB_OUT=$(./aes_encrypt "$PLAINTEXT" | tail -n 1 | grep -oE '[0-9a-fA-F]{32}')
24
25    if [ "${AES_OUT,,}" = "${WB_OUT,,}" ]; then
26        echo " [OK] Output match: $AES_OUT"
27    else
28        echo " [FAIL] Output mismatch!"
29        echo " Base: $AES_OUT"
30        echo " WB  : $WB_OUT"
31        exit 1
32    fi
33 }
34
35 # ---- MAIN ----
36 if [ "$1" == "-random" ]; then
37     NUM_TESTS=5
38     echo " * Cleaning and building base binaries"
39     make clean > /dev/null; make aes aes_whitener > /dev/null
40
41     for i in $(seq 1 $NUM_TESTS); do
42         echo "--- Test $i / $NUM_TESTS ---"
43         KEY=$(generate_random_hex); PLAINTEXT=$(generate_random_hex)
44         run_test "$KEY" "$PLAINTEXT"
45     done
46     echo ""; echo "Successfully passed $NUM_TESTS random tests!"
47 else
48     if [ "$#" -ne 2 ]; then
49         echo "Usage: $0 <key> <plaintext>"
50         echo " or: $0 -random (runs 5 automated tests)"
51         exit 1
52     fi
53
54     KEY="$1"; PLAINTEXT="$2"
55     echo " * Cleaning and building base binaries"
56     make clean > /dev/null; make aes aes_whitener > /dev/null
57     run_test "$KEY" "$PLAINTEXT"
58 fi
```

Listing 12: test_wb.sh: testing bash script to ensure validity of white-box output based on verified AES-128 binary

C Extract of the proof-of-concept

```
1  /* Initialize mixing bijections (random 32-bit bit permutations) */
2  static void init_MIX_BIJECTIONS(void) {
3      for (int r = 0; r < NB_ROUNDS - 1; r++) {
4          for (int c = 0; c < STATE_NB_ROW; c++) {
5              for (int i = 0; i < MIX_BITS; i++)
6                  MIX_POS[r][c][i] = (uint8_t)i;
7
8              for (int i = MIX_BITS - 1; i > 0; i--) {
9                  int j = rand() % (i + 1);
10                 uint8_t tmp = MIX_POS[r][c][i];
11                 MIX_POS[r][c][i] = MIX_POS[r][c][j];
12                 MIX_POS[r][c][j] = tmp;
13             }
14         }
15     }
16 }
17
18 /* Initialize the T_BOXES with pre-computed values */
19 static void init_T_BOXES(uint8_t keys[NB_ROUNDS+1][KEY_SIZE]) {
20     uint8_t shiftedKey[KEY_SIZE];
21
22     /* --- GENERATE TABLES FOR ROUNDS 1 TO 9 --- */
23     for (int r = 1; r <= NB_ROUNDS-1; r++) {
24         memcpy(shiftedKey, keys[r-1], KEY_SIZE);
25         /* copy key into shiftedKey variable */
26         ShiftRows(shiftedKey);
27         /* shuffle the round key bytes to align with the shuffled state
28            bytes */
29
30         for (int i = 0; i < KEY_SIZE; i++) {
31             uint8_t mask_hi = 0;
32             uint8_t mask_lo = 0;
33
34             if (r > 1) { /* if we are not in the first round */
35                 int prev_r = r - 2;
36                 /* get previous round index */
37                 int prev_pos = (i * 5) % KEY_SIZE;
38                 /* compute previous position before ShiftRows */
39                 int prev_col = prev_pos / STATE_NB_ROW;
40                 /* get previous round column */
41                 int prev_row = prev_pos % STATE_NB_ROW;
42                 /* get previous round row */
43                 int prev_final_xor = 3 * prev_col + 2;
44                 /* get previous round final XOR index to retrieve the
45                    mask we need to cancel */
46
47                 /* get the mask for the final XOR */
48                 mask_hi = M_XOR[prev_r][prev_final_xor][prev_row * 2];
49                 /* extract the high 4-bit nibble of the mask */
50                 mask_lo = M_XOR[prev_r][prev_final_xor][prev_row * 2 + 1];
51                 /* extract the low 4-bit nibble of the mask */
52
53             }
54             uint8_t input_mask = (mask_hi << 4) | mask_lo;
55             /* combine the two 4-bit nibbles into a full 8-bit mask */
56
57             for (int x = 0; x < ALL_BYTES_FOR_KEY; x++) {
58                 /* iterate over all 256 possible byte values */
59                 uint8_t real_x = x ^ input_mask;
60                 /* remove the input mask to get the true state byte */
61                 T_BOXES[r-1][i][x] = SBox[real_x ^ shiftedKey[i]];
62                 /* apply AddRoundKey and then SubBytes (S-Box) */
63             }
64         }
65     }
66 }
```

```

50
51 /* --- GENERATE TABLE FOR ROUND 10 (FINAL ROUND) --- */
52 memcpy(shiftedKey, keys[NB_ROUNDS-1], KEY_SIZE);
53 /* copy the Round 9 key */
54 ShiftRows(shiftedKey);
55 /* shuffle the round key bytes to align with the state */
56
57 for (int i = 0; i < KEY_SIZE; i++) {
58     int prev_r = NB_ROUNDS - 2;
59     /* get previous round index (Round 9 is index 8) */
60     int prev_pos = (i * 5) % KEY_SIZE;
61     /* compute previous position before final ShiftRows */
62     int prev_col = prev_pos / STATE_NB_ROW;
63     /* get previous round column */
64     int prev_row = prev_pos % STATE_NB_ROW;
65     /* get previous round row */
66     int prev_final_xor = 3 * prev_col + 2;
67     /* get previous round final XOR index to retrieve the mask we
68     need to cancel */
69
70     uint8_t mask_hi = M_XOR[prev_r][prev_final_xor][prev_row * 2];
71     /* extract the high 4-bit nibble of the mask */
72     uint8_t mask_lo = M_XOR[prev_r][prev_final_xor][prev_row * 2 + 1];
73     /* extract the low 4-bit nibble of the mask */
74     uint8_t input_mask = (mask_hi << 4) | mask_lo;
75     /* combine the two 4-bit nibbles into a full 8-bit mask */
76
77     for (int x = 0; x < ALL_BYTES_FOR_KEY; x++) {
78         /* iterate over all 256 possible byte values */
79         uint8_t real_x = x ^ input_mask;
80         /* remove the input mask to get the true state byte */
81
82         /* apply AddRoundKey(Round 9), SubBytes(S-Box), and final
83         AddRoundKey(Round 10) */
84         T_BOXES[NB_ROUNDS-1][i][x] = SBox[real_x ^ shiftedKey[i]] ^
85             keys[NB_ROUNDS][i];
86     }
87 }
88 memcpy(FINAL_T_BOX, T_BOXES[NB_ROUNDS - 1], sizeof(FINAL_T_BOX));
89 /* save the final round out into a dedicated array */
90 }
91
92 /* Initialize the TY tables with matrix multiplication values */
93 static void init_TY_TABLES(void) {
94     for(int ty = 0; ty < 4; ty++)
95         for(int x = 0; x < ALL_BYTES_FOR_KEY; x++ )
96             TY_TABLES[ty][x] = MatrixMult(ty, x);
97 }
98
99 /* Initialize the CTY tables with the combination of T_BOXES and TY_TABLES
100 with masks */
101 void init_CTY_TABLES(void) {
102     for (int r = 0; r < NB_ROUNDS - 1; r++) {
103         /* loop through rounds 1 to 9 (Round 10 has no MixColumns) */
104         for (int y = 0; y < 4; y++) {
105             /* loop through the 4 rows of the MixColumns matrix */
106             for (int c = 0; c < 4; c++) {
107                 /* loop through the 4 column positions */
108                 int i = c * STATE_NB_ROW + y;
109                 /* only index consumed at runtime for this (y, c) pair
110                 */
111
112                 for (int x = 0; x < ALL_BYTES_FOR_KEY; x++) {
113                     /* iterate over all 256 possible byte values */
114                     uint8_t t = T_BOXES[r][i][x];
115                     /* get the evaluated byte from the T-Box */

```

```

92     uint32_t val = TY_TABLES[y][t];
93     /* multiply by the MixColumns GF(128) coefficient
94        (32-bit result) */
95     val = apply_bit_permutation(val, MIX_POS[r][c]);
96     /* apply per-column mixing bijection */
97
98     uint32_t masked_val = 0;
99     /* accumulator for the fully masked 32-bit value */
100    for(int n=0; n<NB_XOR_NIBBLES; n++) {
101        /* loop through the 8 nibbles (4 bits each) of the
102           32-bit value */
103        uint8_t nib = (val >> (4*(7-n))) & 0xF;
104        /* extract the n-th 4-bit nibble from left to
105           right */
106        nib ^= M_CTY[r][y][n];
107        /* apply the random M_CTY disguise mask to the
108           nibble */
109        masked_val |= ((uint32_t)nib) << (4*(7-n));
110        /* shift the masked nibble back into its proper
111           32-bit position */
112    }
113
114    CTY_TABLES[r][y][c][x] = masked_val;
115    /* save into the compact look-up table */
116
117    }
118
119    }
120
121    }
122
123    }
124
125    }
126
127    }
128
129    }
130
131    }
132
133    }
134
135    }
136
137    }
138
139    }
140
141    }
142
143    }
144
145    }
146
147    }
148
149    }
150
151    }
152
153    }
154
155    }
156
157    }
158
159    }
160
161    }
162
163    }
164
165    }
166
167    }
168
169    }
170
171    }
172
173    }
174
175    }
176
177    }
178
179    }
180
181    }
182
183    }
184
185    }
186
187    }
188
189    }
190
191    }
192
193    }
194
195    }
196
197    }
198
199    }
200
201    }
202
203    }
204
205    }
206
207    }
208
209    }
210
211    }
212
213    }
214
215    }
216
217    }
218
219    }
220
221    }
222
223    }
224
225    }
226
227    }
228
229    }
230
231    }
232
233    }
234
235    }
236
237    }
238
239    }
240
241    }
242
243    }
244
245    }
246
247    }
248
249    }
250
251    }
252
253    }
254
255    }
256
257    }
258
259    }
260
261    }
262
263    }
264
265    }
266
267    }
268
269    }
270
271    }
272
273    }
274
275    }
276
277    }
278
279    }
280
281    }
282
283    }
284
285    }
286
287    }
288
289    }
290
291    }
292
293    }
294
295    }
296
297    }
298
299    }
300
301    }
302
303    }
304
305    }
306
307    }
308
309    }
310
311    }
312
313    }
314
315    }
316
317    }
318
319    }
320
321    }
322
323    }
324
325    }
326
327    }
328
329    }
330
331    }
332
333    }
334
335    }
336
337    }
338
339    }
340
341    }
342
343    }
344
345    }
346
347    }
348
349    }
350
351    }
352
353    }
354
355    }
356
357    }
358
359    }
360
361    }
362
363    }
364
365    }
366
367    }
368
369    }
370
371    }
372
373    }
374
375    }
376
377    }
378
379    }
380
381    }
382
383    }
384
385    }
386
387    }
388
389    }
390
391    }
392
393    }
394
395    }
396
397    }
398
399    }
400
401    }
402
403    }
404
405    }
406
407    }
408
409    }
410
411    }
412
413    }
414
415    }
416
417    }
418
419    }
420
421    }
422
423    }
424
425    }
426
427    }
428
429    }
430
431    }
432
433    }
434
435    }
436
437    }
438
439    }
440
441    }
442
443    }
444
445    }
446
447    }
448
449    }
450
451    }
452
453    }
454
455    }
456
457    }
458
459    }
460
461    }
462
463    }
464
465    }
466
467    }
468
469    }
470
471    }
472
473    }
474
475    }
476
477    }
478
479    }
480
481    }
482
483    }
484
485    }
486
487    }
488
489    }
490
491    }
492
493    }
494
495    }
496
497    }
498
499    }
500
501    }
502
503    }
504
505    }
506
507    }
508
509    }
510
511    }
512
513    }
514
515    }
516
517    }
518
519    }
520
521    }
522
523    }
524
525    }
526
527    }
528
529    }
530
531    }
532
533    }
534
535    }
536
537    }
538
539    }
540
541    }
542
543    }
544
545    }
546
547    }
548
549    }
550
551    }
552
553    }
554
555    }
556
557    }
558
559    }
560
561    }
562
563    }
564
565    }
566
567    }
568
569    }
570
571    }
572
573    }
574
575    }
576
577    }
578
579    }
580
581    }
582
583    }
584
585    }
586
587    }
588
589    }
590
591    }
592
593    }
594
595    }
596
597    }
598
599    }
600
601    }
602
603    }
604
605    }
606
607    }
608
609    }
610
611    }
612
613    }
614
615    }
616
617    }
618
619    }
620
621    }
622
623    }
624
625    }
626
627    }
628
629    }
630
631    }
632
633    }
634
635    }
636
637    }
638
639    }
640
641    }
642
643    }
644
645    }
646
647    }
648
649    }
650
651    }
652
653    }
654
655    }
656
657    }
658
659    }
660
661    }
662
663    }
664
665    }
666
667    }
668
669    }
670
671    }
672
673    }
674
675    }
676
677    }
678
679    }
680
681    }
682
683    }
684
685    }
686
687    }
688
689    }
690
691    }
692
693    }
694
695    }
696
697    }
698
699    }
700
701    }
702
703    }
704
705    }
706
707    }
708
709    }
710
711    }
712
713    }
714
715    }
716
717    }
718
719    }
720
721    }
722
723    }
724
725    }
726
727    }
728
729    }
730
731    }
732
733    }
734
735    }
736
737    }
738
739    }
740
741    }
742
743    }
744
745    }
746
747    }
748
749    }
750
751    }
752
753    }
754
755    }
756
757    }
758
759    }
760
761    }
762
763    }
764
765    }
766
767    }
768
769    }
770
771    }
772
773    }
774
775    }
776
777    }
778
779    }
780
781    }
782
783    }
784
785    }
786
787    }
788
789    }
790
791    }
792
793    }
794
795    }
796
797    }
798
799    }
800
801    }
802
803    }
804
805    }
806
807    }
808
809    }
810
811    }
812
813    }
814
815    }
816
817    }
818
819    }
820
821    }
822
823    }
824
825    }
826
827    }
828
829    }
830
831    }
832
833    }
834
835    }
836
837    }
838
839    }
840
841    }
842
843    }
844
845    }
846
847    }
848
849    }
850
851    }
852
853    }
854
855    }
856
857    }
858
859    }
860
861    }
862
863    }
864
865    }
866
867    }
868
869    }
870
871    }
872
873    }
874
875    }
876
877    }
878
879    }
880
881    }
882
883    }
884
885    }
886
887    }
888
889    }
890
891    }
892
893    }
894
895    }
896
897    }
898
899    }
900
901    }
902
903    }
904
905    }
906
907    }
908
909    }
910
911    }
912
913    }
914
915    }
916
917    }
918
919    }
920
921    }
922
923    }
924
925    }
926
927    }
928
929    }
930
931    }
932
933    }
934
935    }
936
937    }
938
939    }
940
941    }
942
943    }
944
945    }
946
947    }
948
949    }
950
951    }
952
953    }
954
955    }
956
957    }
958
959    }
960
961    }
962
963    }
964
965    }
966
967    }
968
969    }
970
971    }
972
973    }
974
975    }
976
977    }
978
979    }
980
981    }
982
983    }
984
985    }
986
987    }
988
989    }
990
991    }
992
993    }
994
995    }
996
997    }
998
999    }
1000
1001    }
1002
1003    }
1004
1005    }
1006
1007    }
1008
1009    }
1010
1011    }
1012
1013    }
1014
1015    }
1016
1017    }
1018
1019    }
1020
1021    }
1022
1023    }
1024
1025    }
1026
1027    }
1028
1029    }
1030
1031    }
1032
1033    }
1034
1035    }
1036
1037    }
1038
1039    }
1040
1041    }
1042
1043    }
1044
1045    }
1046
1047    }
1048
1049    }
1050
1051    }
1052
1053    }
1054
1055    }
1056
1057    }
1058
1059    }
1060
1061    }
1062
1063    }
1064
1065    }
1066
1067    }
1068
1069    }
1070
1071    }
1072
1073    }
1074
1075    }
1076
1077    }
1078
1079    }
1080
1081    }
1082
1083    }
1084
1085    }
1086
1087    }
1088
1089    }
1090
1091    }
1092
1093    }
1094
1095    }
1096
1097    }
1098
1099    }
1100
1101    }
1102
1103    }
1104
1105    }
1106
1107    }
1108
1109    }
1110
1111    }
1112
1113    }
1114
1115    }
1116
1117    }
1118
1119    }
1120
1121    }
1122
1123    }
1124
1125    }
1126
1127    }
1128
1129    }
1130
1131    }
1132
1133    }
1134
1135    }
1136
1137    }
1138
1139    }
1140
1141    }
1142
1143    }
1144
1145    }
1146
1147    }
1148
1149    }
1150
1151    }
1152
1153    }
1154
1155    }
1156
1157    }
1158
1159    }
1160
1161    }
1162
1163    }
1164
1165    }
1166
1167    }
1168
1169    }
1170
1171    }
1172
1173    }
1174
1175    }
1176
1177    }
1178
1179    }
1180
1181    }
1182
1183    }
1184
1185    }
1186
1187    }
1188
1189    }
1190
1191    }
1192
1193    }
1194
1195    }
1196
1197    }
1198
1199    }
1200
1201    }
1202
1203    }
1204
1205    }
1206
1207    }
1208
1209    }
1210
1211    }
1212
1213    }
1214
1215    }
1216
1217    }
1218
1219    }
1220
1221    }
1222
1223    }
1224
1225    }
1226
1227    }
1228
1229    }
1230
1231    }
1232
1233    }
1234
1235    }
1236
1237    }
1238
1239    }
1240
1241    }
1242
1243    }
1244
1245    }
1246
1247    }
1248
1249    }
1250
1251    }
1252
1253    }
1254
1255    }
1256
1257    }
1258
1259    }
1260
1261    }
1262
1263    }
1264
1265    }
1266
1267    }
1268
1269    }
1270
1271    }
1272
1273    }
1274
1275    }
1276
1277    }
1278
1279    }
1280
1281    }
1282
1283    }
1284
1285    }
1286
1287    }
1288
1289    }
1290
1291    }
1292
1293    }
1294
1295    }
1296
1297    }
1298
1299    }
1300
1301    }
1302
1303    }
1304
1305    }
1306
1307    }
1308
1309    }
1310
1311    }
1312
1313    }
1314
1315    }
1316
1317    }
1318
1319    }
1320
1321    }
1322
1323    }
1324
1325    }
1326
1327    }
1328
1329    }
1330
1331    }
1332
1333    }
1334
1335    }
1336
1337    }
1338
1339    }
1340
1341    }
1342
1343    }
1344
1345    }
1346
1347    }
1348
1349    }
1350
1351    }
1352
1353    }
1354
1355    }
1356
1357    }
1358
1359    }
1360
1361    }
1362
1363    }
1364
1365    }
1366
1367    }
1368
1369    }
1370
1371    }
1372
1373    }
1374
1375    }
1376
1377    }
1378
1379    }
1380
1381    }
1382
1383    }
1384
1385    }
1386
1387    }
1388
1389    }
1390
1391    }
1392
1393    }
1394
1395    }
1396
1397    }
1398
1399    }
1400
1401    }
1402
1403    }
1404
1405    }
1406
1407    }
1408
1409    }
1410
1411    }
1412
1413    }
1414
1415    }
1416
1417    }
1418
1419    }
1420
1421    }
1422
1423    }
1424
1425    }
1426
1427    }
1428
1429    }
1430
1431    }
1432
1433    }
1434
1435    }
1436
1437    }
1438
1439    }
1440
1441    }
1442
1443    }
1444
1445    }
1446
1447    }
1448
1449    }
1450
1451    }
1452
1453    }
1454
1455    }
1456
1457    }
1458
1459    }
1460
1461    }
1462
1463    }
1464
1465    }
1466
1467    }
1468
1469    }
1470
1471    }
1472
1473    }
1474
1475    }
1476
1477    }
1478
1479    }
1480
1481    }
1482
1483    }
1484
1485    }
1486
1487    }
1488
1489    }
1490
1491    }
1492
1493    }
1494
1495    }
1496
1497    }
1498
1499    }
1500
1501    }
1502
1503    }
1504
1505    }
1506
1507    }
1508
1509    }
1510
1511    }
1512
1513    }
1514
1515    }
1516
1517    }
1518
1519    }
1520
1521    }
1522
1523    }
1524
1525    }
1526
1527    }
1528
1529    }
1530
1531    }
1532
1533    }
1534
1535    }
1536
1537    }
1538
1539    }
1540
1541    }
1542
1543    }
1544
1545    }
1546
1547    }
1548
1549    }
1550
1551    }
1552
1553    }
1554
1555    }
1556
1557    }
1558
1559    }
1560
1561    }
1562
1563    }
1564
1565    }
1566
1567    }
1568
1569    }
1570
1571    }
1572
1573    }
1574
1575    }
1576
1577    }
1578
1579    }
1580
1581    }
1582
1583    }
1584
1585    }
1586
1587    }
1588
1589    }
1590
1591    }
1592
1593    }
1594
1595    }
1596
1597    }
1598
1599    }
1600
1601    }
1602
1603    }
1604
1605    }
1606
1607    }
1608
1609    }
1610
1611    }
1612
1613    }
1614
1615    }
1616
1617    }
1618
1619    }
1620
1621    }
1622
1623    }
1624
1625    }
1626
1627    }
1628
1629    }
1630
1631    }
1632
1633    }
1634
1635    }
1636
1637    }
1638
1639    }
1640
1641    }
1642
1643    }
1644
1645    }
1646
1647    }
1648
1649    }
1650
1651    }
1652
1653    }
1654
1655    }
1656
1657    }
1658
1659    }
1660
1661    }
1662
1663    }
1664
1665    }
1666
1667    }
1668
1669    }
1670
1671    }
1672
1673    }
1674
1675    }
1676
1677    }
1678
1679    }
1680
1681    }
1682
1683    }
1684
1685    }
1686
1687    }
1688
1689    }
1690
1691    }
1692
1693    }
1694
1695    }
1696
1697    }
1698
1699    }
1700
1701    }
1702
1703    }
1704
1705    }
1706
1707    }
1708
1709    }
1710
1711    }
1712
1713    }
1714
1715    }
1716
1717    }
1718
1719    }
1720
1721    }
1722
1723    }
1724
1725    }
1726
1727    }
1728
1729    }
1730
1731    }
1732
1733    }
1734
1735    }
1736
1737    }
1738
1739    }
1740
1741    }
1742
1743    }
1744
1745    }
1746
1747    }
1748
1749    }
1750
1751    }
1752
1753    }
1754
1755    }
1756
1757    }
1758
1759    }
1760
1761    }
1762
1763    }
1764
1765    }
1766
1767    }
1768
1769    }
1770
1771    }
1772
1773    }
1774
1775    }
1776
1777    }
1778
1779    }
1780
1781    }
1782
1783    }
1784
1785    }
1786
1787    }
1788
1789    }
1790
1791    }
1792
1793    }
1794
1795    }
1796
1797    }
1798
1799    }
1800
1801    }
1802
1803    }
1804
1805    }
1806
1807    }
1808
1809    }
1810
1811    }
1812
1813    }
1814
1815    }
1816
1817    }
1818
1819    }
1820
1821    }
1822
1823    }
1824
1825    }
1826
1827    }
1828
1829    }
1830
1831    }
1832
1833    }
1834
1835    }
1836
1837    }
1838
1839    }
1840
1841    }
1842
1843    }
1844
1845    }
1846
1847    }
1848
1849    }
1850
1851    }
1852
1853    }
1854
1855    }
1856
1857    }
1858
1859    }
1860
1861    }
1862
1863    }
1864
1865    }
1866
1867    }
1868
1869    }
1870
1871    }
1872
1873    }
1874
1875    }
1876
1877    }
1878
1879    }
1880
1881    }
1882
1883    }
1884
1885    }
1886
1887    }
1888
1889    }
1890
1891    }
1892
1893    }
1894
1895    }
1896
1897    }
1898
1899    }
1900
1901    }
1902
1903    }
1904
1905    }
1906
1907    }
1908
1909    }
1910
1911    }
1912
1913    }
1914
1915    }
1916
1917    }
1918
1919    }
1920
1921    }
1922
1923    }
1924
1925    }
1926
1927    }
1928
1929    }
1930
1931    }
1932
1933    }
1934
1935    }
1936
1937    }
1938
1939    }
1940
1941    }
1942
1943    }
1944
1945    }
1946
1947    }
1948
1949    }
1950
1951    }
1952
1953    }
1954
1955    }
1956
1957    }
1958
1959    }
1960
1961    }
1962
1963    }
1964
1965    }
1966
1967    }
1968
1969    }
1970
1971    }
1972
1973    }
1974
1975    }
1976
1977    }
1978
1979    }
1980
1981    }
1982
1983    }
1984
1985    }
1986
1987    }
1988
1989    }
1990
1991    }
1992
1993    }
1994
1995    }
1996
1997    }
1998
1999    }
2000
2001    }
2002
2003    }
2004
2005    }
2006
2007    }
2008
2009    }
2010
2011    }
2012
2013    }
2014
2015    }
2016
2017    }
2018
2019    }
2020
2021    }
2022
2023    }
2024
2025    }
2026
2027    }
2028
2029    }
2030
2031    }
2032
2033    }
2034
2035    }
2036
2037    }
2038
2039    }
2040
2041    }
2042
2043    }
2044
2045    }
2046
2047    }
2048
2049    }
2050
2051    }
2052
2053    }
2054
2055    }
2056
2057    }
2058
2059    }
2060
2061    }
2062
2063    }
2064
2065    }
2066
2067    }
2068
2069    }
2070
2071    }
2072
2073    }
2074
2075    }
2076
2077    }
2078
2079    }
2080
2081    }
2082
2083    }
2084
2085    }
2086
2087    }
2088
2089    }
2090
2091    }
2092
2093    }
2094
2095    }
2096
2097    }
2098
2099    }
2100
2101    }
2102
2103    }
2104
2105    }
2106
2107    }
2108
2109    }
2110
2111    }
2112
2113    }
2114
2115    }
2116
2117    }
2118
2119    }
2120
2121    }
2122
2123    }
2124
2125    }
2126
2127    }
2128
2129    }
2130
2131    }
2132
2133    }
2134
2135    }
2136
2137    }
2138
2139    }
2140
2141    }
2142
2143    }
2144
2145    }
2146
2147    }
2148
2149    }
2150
2151    }
2152
2153    }
2154
2155    }
2156
2157    }
2158
2159    }
2160
2161    }
2162
2163    }
2164
2165    }
2166
2167    }
2168
2169    }
2170
2171    }
2172
2173    }
2174
2175    }
2176
2177    }
2178
2179    }
2180
2181    }
2182
2183    }
2184
2185    }
2186
2187    }
2188
2189    }
2190
2191    }
2192
2193    }
2194
2195    }
2196
2197    }
2198
2199    }
2200
2201    }
2202
2203    }
2204
2205    }
2206
2207    }
2208
2209    }
2210
2211    }
2212
2213    }
2214
2215    }
2216
2217    }
2218
2219    }
2220
2221    }
2222
2223    }
2224
2225    }
2226
2227    }
2228
2229    }
2230
2231    }
2232
2233    }
2234
2235    }
2236
2237    }
2238
2239    }
2240
2241    }
2242
2243    }
2244
2245    }
2246
2247    }
2248
2249    }
2250
2251    }
2252
2253    }
2254
2255    }
2256
2257    }
2258
2259    }
2260
2261    }
2262
2
```

```

130         int base = xi - 2;
131         /* Node 2 (Root): index of Node 0 for this column
132          */
133         mask_a_ptr = M_XOR[r][base];
134         /* Node 2 input A mask comes from Node 0 output
135          mask */
136         mask_b_ptr = M_XOR[r][base + 1];
137         /* Node 2 input B mask comes from Node 1 output
138          mask */
139         dst = MIX_POS[r][col][src];
140         /* route permuted source bit into its unpermuted
141          destination */
142         out_bit_index = dst;
143     }
144
145     uint8_t mask_a = get_mask_bit(mask_a_ptr, src);
146     uint8_t mask_b = get_mask_bit(mask_b_ptr, src);
147     uint8_t mask_out = get_mask_bit(M_XOR[r][xi], out_bit_index
148     );
149     uint8_t delta = mask_a ^ mask_b ^ mask_out;
150
151     for (int a = 0; a < 2; a++) {
152         for (int b = 0; b < 2; b++) {
153             uint8_t out_bit = (uint8_t)(a ^ b ^ delta);
154             XOR_BIT_TABLES[r][xi][src][a][b] = out_bit ? (1U <<
155             (31 - dst)) : 0U;
156         }
157     }
158 }
159
160 }
161
162 }
163
164 }
165
166 }
167
168 }
169
170 }
171
172 }
173
174 }
175
176 }
177
178 }
179
180 }
181
182 }
183
184 }
185
186 }
187
188 }
189
190 }
191
192 }
193
194 }
195
196 }
197
198 }
199
200 }
201
202 }
203
204 }
205
206 }
207
208 }
209
210 }
211
212 }
213
214 }
215
216 }
217
218 }
219
220 }
221
222 }
223
224 }
225
226 }
227
228 }
229
230 }
231
232 }
233
234 }
235
236 }
237
238 }
239
240 }
241
242 }
243
244 }
245
246 }
247
248 }
249
250 }
251
252 }
253
254 }
255
256 }
257
258 }
259
260 }
261
262 }
263
264 }
265
266 }
267
268 }
269
270 }
271
272 }
273
274 }
275
276 }
277
278 }
279
280 }
281
282 }
283
284 }
285
286 }
287
288 }
289
290 }
291
292 }
293
294 }
295
296 }
297
298 }
299
300 }
301
302 }
303
304 }
305
306 }
307
308 }
309
310 }
311
312 }
313
314 }
315
316 }
317
318 }
319
320 }
321
322 }
323
324 }
325
326 }
327
328 }
329
330 }
331
332 }
333
334 }
335
336 }
337
338 }
339
340 }
341
342 }
343
344 }
345
346 }
347
348 }
349
350 }
351
352 }
353
354 }
355
356 }
357
358 }
359
360 }
361
362 }
363
364 }
365
366 }
367
368 }
369
370 }
371
372 }
373
374 }
375
376 }
377
378 }
379
380 }
381
382 }
383
384 }
385
386 }
387
388 }
389
390 }
391
392 }
393
394 }
395
396 }
397
398 }
399
400 }
401
402 }
403
404 }
405
406 }
407
408 }
409
410 }
411
412 }
413
414 }
415
416 }
417
418 }
419
420 }
421
422 }
423
424 }
425
426 }
427
428 }
429
430 }
431
432 }
433
434 }
435
436 }
437
438 }
439
440 }
441
442 }
443
444 }
445
446 }
447
448 }
449
450 }
451
452 }
453
454 }
455
456 }
457
458 }
459
460 }
461
462 }
463
464 }
465
466 }
467
468 }
469
470 }
471
472 }
473
474 }
475
476 }
477
478 }
479
480 }
481
482 }
483
484 }
485
486 }
487
488 }
489
490 }
491
492 }
493
494 }
495
496 }
497
498 }
499
500 }
501
502 }
503
504 }
505
506 }
507
508 }
509
510 }
511
512 }
513
514 }
515
516 }
517
518 }
519
520 }
521
522 }
523
524 }
525
526 }
527
528 }
529
530 }
531
532 }
533
534 }
535
536 }
537
538 }
539
540 }
541
542 }
543
544 }
545
546 }
547
548 }
549
550 }
551
552 }
553
554 }
555
556 }
557
558 }
559
560 }
561
562 }
563
564 }
565
566 }
567
568 }
569
570 }
571
572 }
573
574 }
575
576 }
577
578 }
579
580 }
581
582 }
583
584 }
585
586 }
587
588 }
589
590 }
591
592 }
593
594 }
595
596 }
597
598 }
599
600 }
601
602 }
603
604 }
605
606 }
607
608 }
609
610 }
611
612 }
613
614 }
615
616 }
617
618 }
619
620 }
621
622 }
623
624 }
625
626 }
627
628 }
629
630 }
631
632 }
633
634 }
635
636 }
637
638 }
639
640 }
641
642 }
643
644 }
645
646 }
647
648 }
649
650 }
651
652 }
653
654 }
655
656 }
657
658 }
659
660 }
661
662 }
663
664 }
665
666 }
667
668 }
669
670 }
671
672 }
673
674 }
675
676 }
677
678 }
679
680 }
681
682 }
683
684 }
685
686 }
687
688 }
689
690 }
691
692 }
693
694 }
695
696 }
697
698 }
699
700 }
701
702 }
703
704 }
705
706 }
707
708 }
709
710 }
711
712 }
713
714 }
715
716 }
717
718 }
719
720 }
721
722 }
723
724 }
725
726 }
727
728 }
729
730 }
731
732 }
733
734 }
735
736 }
737
738 }
739
740 }
741
742 }
743
744 }
745
746 }
747
748 }
749
750 }
751
752 }
753
754 }
755
756 }
757
758 }
759
760 }
761
762 }
763
764 }
765
766 }
767
768 }
769
770 }
771
772 }
773
774 }
775
776 }
777
778 }
779
780 }
781
782 }
783
784 }
785
786 }
787
788 }
789
790 }
791
792 }
793
794 }
795
796 }
797
798 }
799
800 }
801
802 }
803
804 }
805
806 }
807
808 }
809
810 }
811
812 }
813
814 }
815
816 }
817
818 }
819
820 }
821
822 }
823
824 }
825
826 }
827
828 }
829
830 }
831
832 }
833
834 }
835
836 }
837
838 }
839
840 }
841
842 }
843
844 }
845
846 }
847
848 }
849
850 }
851
852 }
853
854 }
855
856 }
857
858 }
859
860 }
861
862 }
863
864 }
865
866 }
867
868 }
869
870 }
871
872 }
873
874 }
875
876 }
877
878 }
879
880 }
881
882 }
883
884 }
885
886 }
887
888 }
889
890 }
891
892 }
893
894 }
895
896 }
897
898 }
899
900 }
901
902 }
903
904 }
905
906 }
907
908 }
909
910 }
911
912 }
913
914 }
915
916 }
917
918 }
919
920 }
921
922 }
923
924 }
925
926 }
927
928 }
929
930 }
931
932 }
933
934 }
935
936 }
937
938 }
939
940 }
941
942 }
943
944 }
945
946 }
947
948 }
949
950 }
951
952 }
953
954 }
955
956 }
957
958 }
959
960 }
961
962 }
963
964 }
965
966 }
967
968 }
969
970 }
971
972 }
973
974 }
975
976 }
977

```

```

182     fprintf(wb_header, "\t{\n");
183     for (int c = 0; c < STATE_NB_ROW; c++) {
184         fprintf(wb_header, "\t\t{");
185         for (int x = 0; x < ALL_BYTES_FOR_KEY; x++) {
186             fprintf(wb_header, "0x%08X%s", CTY_TABLES[r][y][c][x],
187                     COMMA_IF(x + 1 < ALL_BYTES_FOR_KEY));
188         }
189         fprintf(wb_header, "%s\n", COMMA_IF(c + 1 < STATE_NB_ROW));
190     }
191     fprintf(wb_header, "\t}%s\n", COMMA_IF(y + 1 < STATE_NB_ROW));
192 }
193     fprintf(wb_header, "%s\n", COMMA_IF(r + 1 < NB_ROUNDS - 1));
194 }
195     fprintf(wb_header, "};\n\n");
196
197 /* XOR_BIT_TABLES */
198 fprintf(wb_header, "static const uint32_t XOR_BIT_TABLES[%d][%d][%d][2][2] = {\n",
199         NB_ROUNDS - 1, NB_XOR_PER_ROUND, NB_XOR_BITS);
200 for (int r = 0; r < NB_ROUNDS - 1; r++) {
201     fprintf(wb_header, "{\n");
202     for (int x = 0; x < NB_XOR_PER_ROUND; x++) {
203         fprintf(wb_header, "\t{\n");
204         for (int bit = 0; bit < NB_XOR_BITS; bit++) {
205             fprintf(wb_header, "\t\t{\n");
206             for (int a = 0; a < 2; a++) {
207                 fprintf(wb_header, "\t\t\t{\n");
208                 for (int b = 0; b < 2; b++) {
209                     fprintf(wb_header, "0x%08X", XOR_BIT_TABLES[r][x][bit][a][b]);
210                     fprintf(wb_header, "%s", COMMA_IF(b + 1 < 2));
211                 }
212                 fprintf(wb_header, "%s\n", COMMA_IF(a + 1 < 2));
213             }
214             fprintf(wb_header, "\t\t}%s\n", COMMA_IF(bit + 1 < NB_XOR_BITS));
215         }
216         fprintf(wb_header, "\t}%s\n", COMMA_IF(x + 1 < NB_XOR_PER_ROUND));
217     }
218     fprintf(wb_header, "%s\n", COMMA_IF(r + 1 < NB_ROUNDS - 1));
219 }
220     fprintf(wb_header, "};\n\n");

```

Listing 13: Extract of aes_whitener.c: main functions used to generate the white-box header